C++ Standard Library Issues to be moved in Virtual Plenary, June 2021

Doc. no. P2385R0

Date: 2021-05-26

Audience: WG21
Reply to: Jonathan Wakely <lwgchair@gmail.com>

Tentatively Ready Issues

There are 36 issues moved to Tentatively Ready status since the last meeting.

2774. `std::function` construction vs assignment

Section: 20.14.17.3.2 [func.wrap.func.con] **Status:** Tentatively Ready **Submitter:** Barry Revzin **Opened:** 2016-09-14 **Last modified:** 2021-05-20

Priority: 3

View other active issues in [func.wrap.func.con].

View all other issues in [func.wrap.func.con].

Discussion:

I think there's a minor defect in the `std::function` interface. The constructor template is:

```
template <class F> function(F f);
```

while the assignment operator template is

```
template <class F> function& operator=(F&& f);
```

The latter came about as a result of LWG 1288, but that one was dealing with a specific issue that wouldn't have affected the constructor. I think the constructor should also take `f` by forwarding reference, this saves a move in the lvalue/xvalue cases and is also just generally more consistent. Should just make sure that it's stored as `std::decay_t<F>` instead of `F`.

Is there any reason to favor a by-value constructor over a forwarding-reference constructor?

[2019-07-26 Tim provides PR.]

Previous resolution [SUPERSEDED]:

This wording is relative to N4820.

1. Edit 20.14.17.3 [func.wrap.func], class template `function` synopsis, as indicated:

```
namespace std {
  template<class> class function; // not defined
  template<class R, class... ArgTypes> {
  public:
    using result_type = R;

    // 20.14.17.3.2 [func.wrap.func.con], construct/copy/destroy
    function() noexcept;
    function(nullptr_t) noexcept;
    function(const function&);
    function(function&&) noexcept;
    template<class F> function(F&&);

    […]
  };

  […]
}
```

2. Edit 20.14.17.3.2 [func.wrap.func.con] p7-11 as indicated:

```
template<class F> function(F&& f);
```

-7- ~~*Requires:* `F` shall be *Cpp17CopyConstructible*~~Let `FD` be `decay_t<F>`.

-8- ~~*Remarks:* This constructor template shall not participate in overload resolution unless `F`~~ *Constraints:*

(8.1) — `is_same_v<FD, function>` is `false`; and

(8.2) — `FD` is Lvalue-Callable (20.14.17.3 [func.wrap.func]) for argument types `ArgTypes...` and return type `R`.

-?- *Expects:* `FD` meets the *Cpp17CopyConstructible* requirements.

-9- *Ensures:* `!*this` if any of the following hold:

(9.1) — `f` is a null function pointer value.

(9.2) — `f` is a null member pointer value.

(9.3) — ~~`F` is an instance~~ `remove_cvref_t<F>` is a specialization of the `function` class template, and `!f` is `true`.

-10- Otherwise, `*this` targets ~~a copy of `f`~~an object of type `FD` direct-non-list-initialized with ~~`std::move(f)`~~ `std::forward<F>(f)`. [*Note:* Implementations should avoid the use of dynamically allocated memory for small callable objects, for example, where `f` ~~is~~ refers to an object holding only a pointer or reference to an object and a member function pointer. — *end note*]

-11- *Throws:* ~~Shall~~ Does not throw exceptions when ~~`f`~~ `FD` is a function pointer type or a specialization of `reference_wrapper`~~`<T> for some T`~~. Otherwise, may throw `bad_alloc` or any exception thrown by ~~`F`'s copy or move constructor~~ the initialization of the target object.

[2020-11-01; Daniel comments and improves the wording]

The proposed wording should — following the line of Marshall's "Mandating" papers — extract from the *Cpp17CopyConstructible* precondition a corresponding *Constraints:* element and in addition to that the wording should replace old-style elements such as *Expects:* by the recently agreed on elements.

See also the related issue LWG 3493.

Previous resolution [SUPERSEDED]:

This wording is relative to N4868.

1. Edit 20.14.17.3 [func.wrap.func], class template `function` synopsis, as indicated:

```
namespace std {
  template<class> class function; // not defined

  template<class R, class... ArgTypes> {
  public:
    using result_type = R;

    // 20.14.17.3.2 [func.wrap.func.con], construct/copy/destroy
    function() noexcept;
    function(nullptr_t) noexcept;
    function(const function&);
    function(function&&) noexcept;
    template<class F> function(F&&);

    […]
  };

  […]
}
```

2. Edit 20.14.17.3.2 [func.wrap.func.con] as indicated:

```
template<class F> function(F&& f);
```

Let `FD` be `decay_t<F>`.

-8- *Constraints:*

(8.1) — `is_same_v<remove_cvref_t<F>, function>` is `false`,

(8.2) — ~~`FD`~~`F` is Lvalue-Callable (20.14.17.3.1 [func.wrap.func.general]) for argument types `ArgTypes...` and return type `R`,

(8.3) — `is_copy_constructible_v<FD>` is `true`, and

(8.4) — `is_constructible_v<FD, F>` is `true`.

-9- *Preconditions:* ~~`F`~~`FD` meets the *Cpp17CopyConstructible* requirements.

-10- *Postconditions:* `!*this` if any of the following hold:

(10.1) — `f` is a null function pointer value.

(10.2) — `f` is a null member pointer value.

(10.3) — ~~`F` is an instance~~`remove_cvref_t<F>` is a specialization of the function class template, and `!f` is `true`.

-11- Otherwise, `*this` targets ~~a copy of `f`~~an object of type `FD` direct-non-list-initialized with ~~`std::move(f)`~~`std::forward<F>(f)`.

-12- *Throws:* Nothing if ~~`f`~~`FD` is a specialization of `reference_wrapper` or a function pointer type. Otherwise, may throw `bad_alloc` or any exception thrown by ~~`F`'s copy or move constructor~~the initialization of the target object.

-13- *Recommended practice:* Implementations should avoid the use of dynamically allocated memory for small callable objects, for example, where `f` ~~is~~refers to an object holding only a pointer or reference to an object and a member function pointer.

[2021-05-17; Tim comments and revises the wording]

The additional constraints added in the previous wording can induce constraint recursion, as noted in the discussion of LWG 3493. The wording below changes them to *Mandates:* instead to allow this issue to make progress independently of that issue.

The proposed resolution below has been implemented and tested on top of libstdc++.

[2021-05-20; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Edit 20.14.17.3.1 [func.wrap.func.general], class template `function` synopsis, as indicated:

```
namespace std {
  template<class> class function; // not defined

  template<class R, class... ArgTypes> {
  public:
    using result_type = R;

    // 20.14.17.3.2 [func.wrap.func.con], construct/copy/destroy
    function() noexcept;
    function(nullptr_t) noexcept;
    function(const function&);
    function(function&&) noexcept;
    template<class F> function(F&&);

    […]
  };

  […]
}
```

2. Edit 20.14.17.3.2 [func.wrap.func.con] as indicated:

```
template<class F> function(F&& f);
```

Let `FD` be `decay_t<F>`.

-8- *Constraints:*

(8.1) — `is_same_v<remove_cvref_t<F>, function>` is `false`, and

(8.2) — `FD`~~`F`~~ is Lvalue-Callable (20.14.17.3.1 [func.wrap.func.general]) for argument types `ArgTypes...` and return type `R`.

-?- *Mandates:*

(?.1) — `is_copy_constructible_v<FD>` is `true`, and

(?.2) — `is_constructible_v<FD, F>` is `true`.

-9- *Preconditions:* ~~`F`~~`FD` meets the *Cpp17CopyConstructible* requirements.

-10- *Postconditions:* `!*this` is `true` if any of the following hold:

(10.1) — `f` is a null function pointer value.

(10.2) — `f` is a null member pointer value.

(10.3) — ~~`F` is an instance~~`remove_cvref_t<F>` is a specialization of the `function` class template, and `!f` is `true`.

-11- Otherwise, `*this` targets ~~a copy of `f`~~an object of type `FD` direct-non-list-initialized with ~~`std::move(f)`~~`std::forward<F>(f)`.

-12- *Throws:* Nothing if ~~`f`~~`FD` is a specialization of `reference_wrapper` or a function pointer type. Otherwise, may throw `bad_alloc` or any exception thrown by ~~`F`'s copy or move constructor~~the initialization of the target object.

-13- *Recommended practice:* Implementations should avoid the use of dynamically allocated memory for small callable objects, for example, where `f` ~~is~~refers to an object holding only a pointer or reference to an object and a member function pointer.

2818. `"::std::"` everywhere rule needs tweaking

Section: 16.4.2.2 [contents] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2016-11-11 **Last modified:** 2021-05-24

Priority: 2

View all other issues in [contents].

Discussion:

[contents]/3 says

> Whenever a name `x` defined in the standard library is mentioned, the name `x` is assumed to be fully qualified as `::std::x`, unless explicitly described otherwise. For example, if the *Effects* section for library function `F` is described as calling library function `G`, the function `::std::G` is meant.

With the introduction of nested namespaces inside `std`, this rule needs tweaking. For instance, `time_point_cast`'s *Returns* clause says "`time_point<Clock, ToDuration>(duration_cast<ToDuration>(t.time_since_epoch()))`"; that reference to `duration_cast` obviously means `::std::chrono::duration_cast`, not `::std::duration_cast`, which doesn't exist.

[Issues Telecon 16-Dec-2016]

Priority 2; Jonathan to provide wording

[2019 Cologne Wednesday night]

Geoffrey suggested editing 16.4.2.2 [contents]/2 to mention the case when we're defining things in a sub-namespace.

Jonathan to word this.

[2020-02-14, Prague; Walter provides wording]

[2020-10-02; Issue processing telecon: new wording from Jens]

Use "Simplified suggestion" in 13 June 2020 email from Jens.

Previous resolution [SUPERSEDED]:

This wording is relative to N4849.

1. Modify 16.4.2.2 [contents] as indicated:

> -3- ~~Whenever a name `x` defined in the standard library is mentioned, the name `x` is assumed to be fully qualified as `::std::x`, unless explicitly described otherwise. For example, if the *Effects:* element for library function `F` is described as calling library function `G`, the function `::std::G` is meant.~~Let `x` be a name specified by the standard library via a declaration in namespace `std` or in a subnamespace of namespace `std`. Whenever `x` is used as an unqualified name in a further specification, it is assumed to correspond to the same `x` that would be found via unqualified name lookup (6.5.3 [basic.lookup.unqual]) performed at that point of use. Similarly, whenever `x` is used as a qualified name in a further specification, it is assumed to correspond to the same `x` that would be found via qualified name lookup (6.5.5 [basic.lookup.qual]) performed at that point of use. [*Note:* Such lookups can never fail in a well-formed program. — *end note*] [*Example:* If an *Effects:* element for a library function `F` specifies that library function `G` is to be used, the function `::std::G` is intended. — *end example*]

Previous resolution [SUPERSEDED]:

This wording is relative to N4849.

1. Modify 16.4.2.2 [contents] as indicated:

> *[Drafting note: Consider adding a note clarifying that the unqualified lookup does not perform ADL.]*
>
> -3- ~~Whenever a name `x` defined in the standard library is mentioned, the name `x` is assumed to be fully qualified as `::std::x`, unless explicitly described otherwise. For example, if the *Effects:* element for library function `F` is described as calling library function `G`, the function `::std::G` is meant.~~ Whenever an unqualified name `x` is used in the specification of

a declaration `D` in clauses 16-32, its meaning is established as-if by performing unqualified name lookup (6.5.3 [basic.lookup.unqual]) in the context of `D`. Similarly, the meaning of a qualified-id is established as-if by performing qualified name lookup (6.5.5 [basic.lookup.qual]) in the context of `D`. *[Example:* The reference to `is_array_v` in the specification of `std::to_array` (22.3.7.6 [array.creation]) refers to `::std::is_array_v`. -- *end example] [Note:* Operators in expressions 12.2.2.3 [over.match.oper] are not so constrained; see 16.4.6.4 [global.functions]. -- *end note]*

[2020-11-04; Jens provides improved wording]

[2020-11-06; Reflector discussion]

Casey suggests to insert "or Annex D" after "in clauses 16-32". This insertion has been performed during reflector discussions immediately because it seemed editorial.

[2020-11-15; Reflector poll]

Set priority status to Tentatively Ready after seven votes in favour during reflector discussions.

[2020-11-22, Tim Song reopens]

The references to get in 24.5.4.1 [range.subrange.general] and 24.7.16.2 [range.elements.view] need to be qualified as they would otherwise refer to `std::ranges::get` instead of `std::get`. Additionally, 16.3.3.2 [expos.only.func] needs to clarify that the lookup there also takes place from within namespace `std`.

Previous resolution [SUPERSEDED]:

This wording is relative to N4868.

1. Modify 16.4.2.2 [contents] as indicated:

-3- ~~Whenever a name `x` defined in the standard library is mentioned, the name `x` is assumed to be fully qualified as `::std::x`, unless explicitly described otherwise. For example, if the *Effects:* element for library function `F` is described as calling library function `G`, the function `::std::G` is meant.~~ Whenever an unqualified name `x` is used in the specification of a declaration `D` in clauses 16-32 or Annex D, its meaning is established as-if by performing unqualified name lookup (6.5.3 [basic.lookup.unqual]) in the context of `D`. [*Note ?:* Argument-dependent lookup is not performed. — *end note*] Similarly, the meaning of a qualified-id is established as-if by performing qualified name lookup (6.5.5 [basic.lookup.qual]) in the context of `D`. [*Example:* The reference to `is_array_v` in the specification of `std::to_array` (22.3.7.6 [array.creation]) refers to `::std::is_array_v`. — *end example*] [*Note ?:* Operators in expressions (12.2.2.3 [over.match.oper]) are not so constrained; see 16.4.6.4 [global.functions]. — *end note*]

2. Remove 29.11.3.2 [fs.req.namespace] in its entirety:

~~**29.11.3.2 Namespaces and headers [fs.req.namespace]**~~

~~-1- Unless otherwise specified, references to entities described in subclause 29.11 [filesystems] are assumed to be qualified with `::std::filesystem::`.~~

[2021-05-20; Jens Maurer provides an updated proposed resolution]

[2021-05-23; Daniel provides some additional tweaks to the updated proposed resolution]

[2021-05-24; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 16.3.3.2 [expos.only.func] as indicated:

-2- The following are defined for exposition only to aid in the specification of the library:

```
namespace std {
  template<class T> constexpr decay_t<T> decay-copy(T&& v)
      noexcept(is_nothrow_convertible_v<T, decay_t<T>>) // exposition only
    { return std::forward<T>(v); }

  constexpr auto synth-three-way =
    []<class T, class U>(const T& t, const U& u)
      requires requires {
      { t < u } -> boolean-testable;
      { u < t } -> boolean-testable;
      }
    {
      if constexpr (three_way_comparable_with<T, U>) {
        return t <=> u;
      } else {
        if (t < u) return weak_ordering::less;
```

```
            if (u < t) return weak_ordering::greater;
            return weak_ordering::equivalent;
          }
        };

      template<class T, class U=T>
      using synth-three-way-result = decltype(synth-three-way(declval<T&>(), declval<U&>()));
    }
```

2. Modify 16.4.2.2 [contents] as indicated:

> -3- ~~Whenever a name `x` defined in the standard library is mentioned, the name `x` is assumed to be fully qualified as `::std::x`, unless explicitly described otherwise. For example, if the *Effects:* element for library function `E` is described as calling library function `G`, the function `::std::G` is meant.~~ Whenever an unqualified name `x` is used in the specification of a declaration `D` in clauses 16-32 or Annex D, its meaning is established as-if by performing unqualified name lookup (6.5.3 [basic.lookup.unqual]) in the context of `D`. [*Note ?:* Argument-dependent lookup is not performed. — *end note*] Similarly, the meaning of a *qualified-id* is established as-if by performing qualified name lookup (6.5.5 [basic.lookup.qual]) in the context of `D`. [*Example:* The reference to `is_array_v` in the specification of `std::to_array` (22.3.7.6 [array.creation]) refers to `::std::is_array_v`. — *end example*] [*Note ?:* Operators in expressions (12.2.2.3 [over.match.oper]) are not so constrained; see 16.4.6.4 [global.functions]. — *end note*]

3. Modify 24.5.4.1 [range.subrange.general] as indicated:

```
template<class T>
  concept pair-like =              // exposition only
    !is_reference_v<T> && requires(T t) {
      typename tuple_size<T>::type; // ensures tuple_size<T> is complete
      requires derived_from<tuple_size<T>, integral_constant<size_t, 2>>;
      typename tuple_element_t<0, remove_const_t<T>>;
      typename tuple_element_t<1, remove_const_t<T>>;
      { std::get<0>(t) } -> convertible_to<const tuple_element_t<0, T>&>;
      { std::get<1>(t) } -> convertible_to<const tuple_element_t<1, T>&>;
    };
```

4. Modify 24.7.16.2 [range.elements.view] as indicated:

```
template<class T, size_t N>
  concept has-tuple-element = // exposition only
    requires(T t) {
      typename tuple_size<T>::type;
      requires N <tuple_size_v<T>;
      typename tuple_element_t<N, T>;
      { std::get<N>(t) } -> convertible_to<const tuple_element_t<N, T>&>;
    };
```

5. Modify 24.7.16.3 [range.elements.iterator] as indicated:

> -2- The member *typedef-name* `iterator_category` is defined if and only if `Base` models `forward_range`. In that case, `iterator_category` is defined as follows: […]
>
> (2.1) — If `std::get<N>(*current_)` is an rvalue, `iterator_category` denotes `input_iterator_tag`.
>
> […]

```
static constexpr decltype(auto) get-element(const iterator_t<Base>& i); // exposition only
```

> -3- *Effects:* Equivalent to:

```
if constexpr (is_reference_v<range_reference_t<Base>>) {
  return std::get<N>(*i);
} else {
  using E = remove_cv_t<tuple_element_t<N, range_reference_t<Base>>>;
  return static_cast<E>(std::get<N>(*i));
}
```

6. Remove 29.11.3.2 [fs.req.namespace] in its entirety:

> ~~**29.11.3.2 Namespaces and headers [fs.req.namespace]**~~
>
> ~~-1- Unless otherwise specified, references to entities described in subclause 29.11 [filesystems] are assumed to be qualified with `::std::filesystem::`.~~

2997. LWG 491 and the specification of `{forward_,}list::unique`

Section: 22.3.10.5 [list.ops], 22.3.9.6 [forwardlist.ops] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2017-07-07 **Last modified:** 2021-05-21

Priority: 3

View all other issues in [list.ops].

Discussion:

There are various problems with the specification of `list::unique` and its `forward_list` counterpart, some of which are obvious even on cursory inspection:

- It refers to the identifiers `first` and `last`, which aren't even defined.
- It uses `i - 1` on non-random-access iterators - in the case of `forward_list`, on forward-only iterators.
- The resolution of LWG 202, changing the specification of `std::unique` to require an equivalence relation and adjusting the order of comparison, weren't applied to these member functions.

LWG 491, which pointed out many of those problems with the specification of `list::unique`, was closed as NAD with the rationale that

> "All implementations known to the author of this Defect Report comply with these assumption", and "no impact on current code is expected", i.e. there is no evidence of real-world confusion or harm.

That implementations somehow managed to do the right thing in spite of obviously defective standardese doesn't seem like a good reason to not fix the defects.

[2017-07 Toronto Tuesday PM issue prioritization]

Priority 3; by the way, there's general wording in 25.2 [algorithms.requirements] p10 that lets us specify iterator arithmetic as if we were using random access iterators.

[2017-07-11 Tim comments]

I drafted the P/R fully aware of the general wording in 25.2 [algorithms.requirements] p10. However, that general wording is limited to Clause 28, so to make use of the shorthand permitted by that wording, we would need additional wording importing it to these subclauses.

Moreover, that general wording only defines `a+n` and `b-a`; it notably doesn't define `a-n`, which is needed here. And one cannot merely define `a-n` as `a+(-n)` since that has undefined behavior for forward iterators.

Previous resolution [SUPERSEDED]:

This wording is relative to N4659.

1. Edit 22.3.10.5 [list.ops] as indicated:

```
void unique();
template <class BinaryPredicate> void unique(BinaryPredicate binary_pred);
```

-?- *Requires:* The comparison function shall be an equivalence relation.

-19- *Effects:* If `empty()`, has no effects. Otherwise, e~~E~~rases all but the first element from every consecutive group of ~~equal~~equivalent elements referred to by the iterator `i` in the range ~~`[first + 1, last)`~~ `[next(begin()), end())` for which ~~`*i == *(i - 1)`~~`*j == *i` (for the version of `unique` with no arguments) or ~~`pred(*i, *(i - 1))`~~`pred(*j, *i)` (for the version of `unique` with a predicate argument) holds, where `j` is an iterator in `[begin(), end())` such that `next(j) == i`. Invalidates only the iterators and references to the erased elements.

-20- *Throws:* Nothing unless an exception is thrown by ~~`*i == *(i-1)` or `pred(*i, *(i - 1))`~~ the equality comparison or the predicate.

-21- *Complexity:* If ~~the range `[first, last)` is not empty~~`!empty()`, exactly ~~`(last - first) - 1`~~`size() - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

2. Edit 22.3.9.6 [forwardlist.ops] as indicated:

```
void unique();
template <class BinaryPredicate> void unique(BinaryPredicate binary_pred);
```

-?- *Requires:* The comparison function shall be an equivalence relation.

-16- *Effects:* If `empty()`, has no effects. Otherwise, e~~E~~rases all but the first element from every consecutive group of ~~equal~~equivalent elements referred to by the iterator `i` in the range ~~`[first + 1, last)`~~ `[next(begin()), end())` for which ~~`*i == *(i - 1)`~~`*j == *i` (for the version with no arguments) or ~~`pred(*i, *(i - 1))`~~`pred(*j, *i)` (for the version with a predicate argument) holds, where `j` is an iterator in `[begin(), end())` such that `next(j) == i`. Invalidates only the iterators and references to the erased elements.

-17- *Throws:* Nothing unless an exception is thrown by the equality comparison or the predicate.

-18- *Complexity:* If ~~the range `[first, last)` is not empty~~`!empty()`, exactly ~~`(last - first) - 1`~~`distance(begin(), end()) - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

[2021-02-21 Tim redrafts]

The wording below incorporates editorial pull request 4465.

[2021-05-21; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Edit 22.3.10.5 [list.ops] as indicated:

> -1- Since lists allow fast insertion and erasing from the middle of a list, certain operations are provided specifically for them.[222] In this subclause, arguments for a template parameter named `Predicate` or `BinaryPredicate` shall meet the corresponding requirements in 25.2 [algorithms.requirements]. The semantics of `i + n` and `i - n`, where `i` is an iterator into the list and `n` is an integer, are the same as those of `next(i, n)` and `prev(i, n)`, respectively. For `merge` and `sort`, the definitions and requirements in 25.8 [alg.sorting] apply.
>
> […]
>
> ```
> void unique();
> template<class BinaryPredicate> void unique(BinaryPredicate binary_pred);
> ```
>
> -?- Let `binary_pred` be `equal_to<>{}` for the first overload.
>
> -?- *Preconditions:* `binary_pred` is an equivalence relation.
>
> -20- *Effects:* Erases all but the first element from every consecutive group of ~~equal~~equivalent elements. That is, for a nonempty list, erases all elements referred to by the iterator `i` in the range ~~`[first + 1, last)`~~`[begin() + 1, end())` for which ~~`*i == *(i-1)` (for the version of `unique` with no arguments) or~~ `binary_pred(*i, *(i - 1))` is true~~(for the version of `unique` with a predicate argument) holds~~. Invalidates only the iterators and references to the erased elements.
>
> -21- *Returns*: The number of elements erased.
>
> -22- *Throws:* Nothing unless an exception is thrown by ~~`*i == *(i-1)` or `pred(*i, *(i - 1))`~~the predicate.
>
> -23- *Complexity:* If ~~the range `[first, last)` is not empty~~`empty()` is false, exactly ~~`(last - first) - 1`~~`size() - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

2. Edit 22.3.9.6 [forwardlist.ops] as indicated:

> -1- In this subclause, arguments for a template parameter named `Predicate` or `BinaryPredicate` shall meet the corresponding requirements in 25.2 [algorithms.requirements]. The semantics of `i + n`, where `i` is an iterator into the list and `n` is an integer, are the same as those of `next(i, n)`. The expression `i - n`, where `i` is an iterator into the list and `n` is an integer, means an iterator `j` such that `j + n == i` is `true`. For `merge` and `sort`, the definitions and requirements in 25.8 [alg.sorting] apply.
>
> […]
>
> ```
> void unique();
> template<class BinaryPredicate> void unique(BinaryPredicate binary_pred);
> ```
>
> -?- Let `binary_pred` be `equal_to<>{}` for the first overload.
>
> -?- *Preconditions:* `binary_pred` is an equivalence relation.
>
> -18- *Effects:* Erases all but the first element from every consecutive group of ~~equal~~equivalent elements. That is, for a nonempty list, erases all elements referred to by the iterator `i` in the range ~~`[first + 1, last)`~~`[begin() + 1, end())` for which ~~`*i == *(i-1)` (for the version with no arguments) or~~ `binary_pred(*i, *(i - 1))` is `true` ~~(for the version with a predicate argument) holds~~. Invalidates only the iterators and references to the erased elements.
>
> -19- *Returns*: The number of elements erased.
>
> -20- *Throws:* Nothing unless an exception is thrown by ~~the equality comparison or~~ the predicate.
>
> -21- *Complexity:* If ~~the range `[first, last)` is not empty~~`empty()` is `false`, exactly ~~`(last - first) - 1`~~`distance(begin(), end()) - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

3410. `lexicographical_compare_three_way` is overspecified

Section: 25.8.12 [alg.three.way] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2020-02-27 **Last modified:** 2021-05-26

Priority: 3

View all other issues in [alg.three.way].

Discussion:

25.8.12 [alg.three.way]/2 specifies the effects of the `lexicographical_compare_three_way` algorithm via a large "equivalent to" codeblock. This codeblock specifies one more iterator comparison than necessary when the first input sequence is greater than the second, and two more than necessary in other cases. Requiring unnecessary work is the antithesis of C++.

[2020-03-29 Issue Prioritization]

Priority to 3 after reflector discussion.

[2021-05-19 Tim adds wording]

The wording below simply respecifies the algorithm in words. It seems pointless to try to optimize the code when the reading in the discussion above would entirely rule out things like `memcmp` optimizations for arbitrary contiguous iterators of bytes.

[2021-05-26; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 25.8.12 [alg.three.way] as indicated:

```
template<class InputIterator1, class InputIterator2, class Cmp>
  constexpr auto
    lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                      InputIterator2 b2, InputIterator2 e2,
                                      Cmp comp)
        -> decltype(comp(*b1, *b2));
```

-?- Let *N* be min(`e1 - b1`, `e2 - b2`). Let *E(n)* be `comp(*(b1 + n), *(b2 + n))`.

-1- *Mandates:* `decltype(comp(*b1, *b2))` is a comparison category type.

-?- *Returns: E(i)*, where *i* is the smallest integer in [0, *N*) such that *E(i)* `!= 0` is `true`, or `(e1 - b1) <=> (e2 - b2)` if no such integer exists.

-?- *Complexity*: At most *N* applications of `comp`.

~~-2- *Effects:* Lexicographically compares two ranges and produces a result of the strongest applicable comparison category type. Equivalent to:~~

```
~~for ( ; b1 != e1 && b2 != e2; void(++b1), void(++b2) )~~
~~  if (auto cmp = comp(*b1,*b2); cmp != 0)~~
~~    return cmp;~~
~~return b1 != e1 ? strong_ordering::greater :~~
~~       b2 != e2 ? strong_ordering::less :~~
~~                  strong_ordering::equal;~~
```

3430. `std::fstream` & co. should be constructible from `string_view`

Section: 29.9.1 [fstream.syn] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2020-04-15 **Last modified:** 2021-05-21

Priority: 3

Discussion:

We have:

```
basic_fstream(const char*, openmode);
basic_fstream(const filesystem::path::value_type*, openmode); // wide systems only
basic_fstream(const string&, openmode);
basic_fstream(const filesystem::path&, openmode);
```

I think the omission of a `string_view` overload was intentional, because the underlying OS call (such as `fopen`) needs a NTBS. We wanted the allocation required to turn a `string_view` into an NTBS to be explicitly requested by the user. But then we added the `path` overload, which is callable with a `string_view`. Converting to a `path` is more expensive than converting to `std::string`, because a path has to *at least* construct a `basic_string`, and potentially also does an encoding conversion, parses the path, and potentially allocates a sequence of `path` objects for the path components.

This means the simpler, more obvious code is slower and uses more memory:

```
string_view sv = "foo.txt";
fstream f1(sv); // bad
fstream f2(string(sv)); // good
```

We should just allow passing a `string_view` directly, since it already compiles but doesn't do what anybody expects or wants.

Even with a `string_view` overload, passing types like `const char16_t*` or `u32string_view` will still implicitly convert to `filesystem::path`, but that seems reasonable. In those cases the encoding conversion is necessary. For Windows we support construction from `const wchar_t*` but not from `wstring` or `wstring_view`, which means those types will convert to `filesystem::path`. That seems suboptimal, so we might also want to add `wstring` and `wstring_view` overloads for "wide systems only", as per 29.9.1 [fstream.syn] p3.

Daniel:

LWG 2883 has a more general view on that but does not consider potential cost differences in the presence of `path` overloads (Which didn't exist at this point yet).

[2020-05-09; Reflector prioritization]

Set priority to 3 after reflector discussions.

[2020-08-10; Jonathan comments]

An alternative fix would be to retain the original design and not allow construction from a `string_view`. The `path` parameters could be changed to template parameters which are constrained to be exactly `path`, and not things like `string_view` which can convert to `path`.

[2020-08-21; Issue processing telecon: send to LEWG]

Just adding support for `string_view` doesn't prevent expensive conversions from other types that convert to `path`. Preference for avoiding all expensive implicit conversions to `path`, maybe via abbreviated function templates:

```
basic_fstream(same_as<filesystem::path> auto const&, openmode);
```

It's possible `path_view` will provide a better option at some point.

It was noted that 2676 did intentionally allow conversions from "strings of character types `wchar_t`, `char16_t`, and `char32_t`". Those conversions don't need to be implicit for that to be supported.

[2020-09-11; Tomasz comments and provides wording]

During the LEWG 2020-08-24 telecon the LEWG provided following guidance on the issue:

> We took one poll (exact wording in the notes) to constrain the constructor which takes `filesystem::path` to only take `filesystem::path` and not things convertible to it, but only 9 out of 26 people present actually voted. Our interpretation: LWG should go ahead with making this change. There is still plenty of time for someone who hasn't yet commented on this to bring it up even if it is in a tentatively ready state. It would be nice to see a paper to address the problem of the templated path constructor, but no one has yet volunteered to do so. Note: the issue description is now a misnomer, as adding a `string_view` constructor is no longer being considered at this time.

The proposed P/R follows original LWG proposal and makes the `path` constructor of the `basic_*fstreams` "`explicit`". To adhere to current policy, we refrain from use of requires clauses and abbreviated function syntax, and introduce a *Constraints* element.

[2021-05-21; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4861.

1. Modify 29.9.3 [ifstream], class template `basic_ifstream` synopsis, as indicated:

```
[…]
explicit basic_ifstream(const string& s,
                        ios_base::openmode mode = ios_base::in);
template<class T>
explicit basic_ifstream(const filesystem::pathT& s,
                        ios_base::openmode mode = ios_base::in);
[…]
```

2. Modify 29.9.3.2 [ifstream.cons] as indicated:

```
explicit basic_ifstream(const string& s,
                        ios_base::openmode mode = ios_base::in);
```

-?- *Effects:* Equivalent to: `basic_ifstream(s.c_str(), mode);`

```
template<class T>
explicit basic_ifstream(const filesystem::pathT& s,
                        ios_base::openmode mode = ios_base::in);
```

-?- *Constraints:* `is_same_v<T, filesystem::path>` is `true`.

-3- *Effects:* Equivalent to: `basic_ifstream(s.c_str(), mode);`.

3. Modify 29.9.4 [ofstream], class template `basic_ofstream` synopsis, as indicated:

```
[…]
explicit basic_ofstream(const string& s,
                        ios_base::openmode mode = ios_base::out);
template<class T>
explicit basic_ofstream(const filesystem::pathT& s,
                        ios_base::openmode mode = ios_base::out);
[…]
```

4. Modify 29.9.4.2 [ofstream.cons] as indicated:

```
explicit basic_ofstream(const string& s,
                        ios_base::openmode mode = ios_base::out);
```

-?- *Effects:* Equivalent to: `basic_ofstream(s.c_str(), mode);`

```
template<class T>
explicit basic_ofstream(const filesystem::pathT& s,
                        ios_base::openmode mode = ios_base::out);
```

-?- *Constraints:* `is_same_v<T, filesystem::path>` is `true`.

-3- *Effects:* Equivalent to: `basic_ofstream(s.c_str(), mode)`.

5. Modify 29.9.5 [fstream], class template `basic_fstream` synopsis, as indicated:

```
[…]
explicit basic_fstream(
  const string& s,
  ios_base::openmode mode = ios_base::in | ios_base::out);
template<class T>
explicit basic_fstream(
  const filesystem::pathT& s,
  ios_base::openmode mode = ios_base::in | ios_base::out);
[…]
```

6. Modify 29.9.5.2 [fstream.cons] as indicated:

```
explicit basic_fstream(
  const string& s,
  ios_base::openmode mode = ios_base::in | ios_base::out);
```

-?- *Effects:* Equivalent to: `basic_fstream(s.c_str(), mode)`.

```
template<class T>
explicit basic_fstream(
  const filesystem::pathT& s,
  ios_base::openmode mode = ios_base::in | ios_base::out);
```

-?- *Constraints:* `is_same_v<T, filesystem::path>` is `true`.

-3- *Effects:* Equivalent to: `basic_fstream(s.c_str(), mode)`.

3462. §[formatter.requirements]: Formatter requirements forbid use of `fc.arg()`

Section: 20.20.5.1 [formatter.requirements] **Status:** Tentatively Ready **Submitter:** Alberto Barbati **Opened:** 2020-06-30 **Last modified:** 2021-05-24

Priority: 3

Discussion:

The requirements on the expression `f.format(t, fc)` in [tab:formatter] say

> Formats `t` according to the specifiers stored in `*this`, writes the output to `fc.out()` and returns an iterator past the end of the output range. The output shall only depend on `t`, `fc.locale()`, and the range `[pc.begin(), pc.end())` from the last call to `f.parse(pc)`.

Strictly speaking, this wording effectively forbids `f.format(t, fc)` from calling `fc.arg(n)`, whose motivation is precisely to allow a formatter to rely on arguments different from `t`. According to this interpretation, there's no conforming way to implement the `"{ arg-id }"` form of the *width* and *precision* fields of standard format specifiers. Moreover, the formatter described in the example if paragraph 20.20.5.4 [format.context]/8 would also be non-conforming.

[2020-07-12; Reflector prioritization]

Set priority to 3 after reflector discussions.

Previous resolution [SUPERSEDED]:

This wording is relative to N4861.

1. Modify 20.20.5.1 [formatter.requirements], Table [tab:formatter], as indicated:

Table 67: *Formatter* requirements [tab:formatter]

Expression	Return type	Requirement
…		
`f.format(t, fc)`	`FC::iterator`	Formats `t` according to the specifiers stored in `*this`, writes the output to `fc.out()` and returns an iterator past the end of the output range. The output shall only depend on `t`, `fc.locale()`, ~~and~~ the range `[pc.begin(), pc.end())` from the last call to `f.parse(pc)`, and `fc.arg(n)`, where `n` is a `size_t` index value that has been validated with a call to `pc.check_arg_id(n)` in the last call to `f.parse(pc)`.
…		

[2021-05-20 Tim comments and updates wording]

During reflector discussion Victor said that the formatter requirements should allow dependency on any of the format arguments in the context. The wording below reflects that.

[2021-05-24; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 20.20.5.1 [formatter.requirements], Table [tab:formatter], as indicated:

Table 67: *Formatter* requirements [tab:formatter]

Expression	Return type	Requirement
…		
`f.format(t, fc)`	`FC::iterator`	Formats `t` according to the specifiers stored in `*this`, writes the output to `fc.out()` and returns an iterator past the end of the output range. The output shall only depend on `t`, `fc.locale()`, `fc.arg(n)` for any value `n` of type `size_t`, and the range `[pc.begin(), pc.end())` from the last call to `f.parse(pc)`.
…		

3481. `viewable_range` mishandles lvalue move-only views

Section: 24.4.5 [range.refinements] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2020-08-29 **Last modified:** 2021-05-24

Priority: 2

View all other issues in [range.refinements].

Discussion:

The `viewable_range` `concept` (24.4.5 [range.refinements]) and the `views:all` range adaptor (24.7.4 [range.all]) are duals: `viewable_range` is intended to admit exactly types `T` for which `views::all(declval<T>())` is well-formed. (Recall that `views::all(`*`meow`*`)` is a prvalue whose type models `view` when it is well-formed.) Before the addition of move-only view types to the design, this relationship was in place (modulo an incredibly pathological case: a `volatile` value of a view type with `volatile`-qualified `begin` and `end` models `viewable_range` but is rejected by `views::all` unless it also has a `volatile`-qualified copy constructor and copy assignment operator). Adding move-only views to the design punches a bigger hole, however: `viewable_range` admits lvalues of move-only view types for which `views::all` is ill-formed because these lvalues cannot be decay-copied.

It behooves us to restore the correspondence between `viewable_range` and `views::all` so that instantiations of components constrained with `viewable_range` (which often appears indirectly as `views::all_t<R>` in deduction guides) continue to be well-formed when the constraints are satisfied.

[2020-09-06; Reflector prioritization]

Set priority to 2 during reflector discussions.

[2021-05-24; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4861.

1. Modify 24.4.5 [range.refinements] as indicated:

-5- The `viewable_range` concept specifies the requirements of a `range` type that can be converted to a `view` safely.

```
template<class T>
  concept viewable_range =
    range<T> && (borrowed_range<T> || view<remove_cvref_t<T>>);
    ((view<remove_cvref_t<T>> && constructible_from<remove_cvref_t<T>, T>) ||
    (!view<remove_cvref_t<T>> && borrowed_range<T>));
```

3506. Missing allocator-extended constructors for `priority_queue`

Section: 22.6.5 [priority.queue] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2020-11-21 **Last modified:** 2021-02-26

Priority: 3

View other active issues in [priority.queue].

View all other issues in [priority.queue].

Discussion:

`priority_queue` has two constructor templates taking a pair of input iterators in addition to a comparator and a container, but it does not have allocator-extended constructors corresponding to these constructor templates:

```
template<class InputIterator>
  priority_queue(InputIterator first, InputIterator last, const Compare& x,
                 const Container&);
template<class InputIterator>
  priority_queue(InputIterator first, InputIterator last,
                 const Compare& x = Compare(), Container&& = Container());
```

[2020-11-29; Reflector prioritization]

Set priority to 3 during reflector discussions. It has been pointed out that this issue is related to LWG 1199, LWG 2210, and LWG 2713.

[2021-02-17 Tim adds PR]

[2021-02-26; LWG telecon]

Set status to Tentatively Ready after discussion and poll.

F A N
11 0 0

Proposed resolution:

This wording is relative to N4878.

1. Add the following paragraph at the end of 22.6.1 [container.adaptors.general]:

   -6- The exposition-only alias template *iter-value-type* defined in 22.3.1 [sequences.general] may appear in deduction guides for container adaptors.

2. Modify 22.6.5 [priority.queue], class template `priority_queue` synopsis, as indicated:

```
namespace std {
  template<class T, class Container = vector<T>,
           class Compare = less<typename Container::value_type>>
  class priority_queue {

  // […]

  public:
    priority_queue() : priority_queue(Compare()) {}
    explicit priority_queue(const Compare& x) : priority_queue(x, Container()) {}
    priority_queue(const Compare& x, const Container&);
    priority_queue(const Compare& x, Container&&);
    template<class InputIterator>
      priority_queue(InputIterator first, InputIterator last, const Compare& x,
                     const Container&);
    template<class InputIterator>
      priority_queue(InputIterator first, InputIterator last,
                     const Compare& x = Compare(), Container&& = Container());
    template<class Alloc> explicit priority_queue(const Alloc&);
    template<class Alloc> priority_queue(const Compare&, const Alloc&);
    template<class Alloc> priority_queue(const Compare&, const Container&, const Alloc&);
    template<class Alloc> priority_queue(const Compare&, Container&&, const Alloc&);
    template<class Alloc> priority_queue(const priority_queue&, const Alloc&);
    template<class Alloc> priority_queue(priority_queue&&, const Alloc&);
    template<class InputIterator, class Alloc>
      priority_queue(InputIterator, InputIterator, const Alloc&);
    template<class InputIterator, class Alloc>
      priority_queue(InputIterator, InputIterator, const Compare&, const Alloc&);
    template<class InputIterator, class Alloc>
      priority_queue(InputIterator, InputIterator, const Compare&, const Container&, const Alloc&);
    template<class InputIterator, class Alloc>
      priority_queue(InputIterator, InputIterator, const Compare&, Container&&, const Alloc&);

  // […]

  };

  template<class Compare, class Container>
    priority_queue(Compare, Container)
      -> priority_queue<typename Container::value_type, Container, Compare>;

  template<class InputIterator,
           class Compare = less<typename iterator_traits<InputIterator>::value_type>,
           class Container = vector<typename iterator_traits<InputIterator>::value_type>>
    priority_queue(InputIterator, InputIterator, Compare = Compare(), Container = Container())
      -> priority_queue<typename iterator_traits<InputIterator>::value_type, Container, Compare>;

  template<class Compare, class Container, class Allocator>
    priority_queue(Compare, Container, Allocator)
      -> priority_queue<typename Container::value_type, Container, Compare>;

  template<class InputIterator, class Allocator>
```

```
  priority_queue(InputIterator, InputIterator, Allocator)
    -> priority_queue<iter-value-type<InputIterator>,
                      vector<iter-value-type<InputIterator>, Allocator>,
                      less<iter-value-type<InputIterator>>>;

  template<class InputIterator, class Compare, class Allocator>
  priority_queue(InputIterator, InputIterator, Compare, Allocator)
    -> priority_queue<iter-value-type<InputIterator>,
                      vector<iter-value-type<InputIterator>, Allocator>, Compare>;

  template<class InputIterator, class Compare, class Container, class Allocator>
  priority_queue(InputIterator, InputIterator, Compare, Container, Allocator)
    -> priority_queue<typename Container::value_type, Container, Compare>;

  // […]
}
```

3. Add the following paragraphs to 22.6.5.3 [priqueue.cons.alloc]:

```
template<class InputIterator, class Alloc>
  priority_queue(InputIterator first, InputIterator last, const Alloc& a);
```

-?- *Effects:* Initializes `c` with `first` as the first argument, `last` as the second argument, and `a` as the third argument, and value-initializes `comp`; calls `make_heap(c.begin(), c.end(), comp)`.

```
template<class InputIterator, class Alloc>
  priority_queue(InputIterator first, InputIterator last, const Compare& compare, const Alloc& a);
```

-?- *Effects:* Initializes `c` with `first` as the first argument, `last` as the second argument, and `a` as the third argument, and initializes `comp` with `compare`; calls `make_heap(c.begin(), c.end(), comp)`.

```
template<class InputIterator, class Alloc>
  priority_queue(InputIterator first, InputIterator last, const Compare& compare, const Container& cont, const Alloc& a);
```

-?- *Effects:* Initializes `c` with `cont` as the first argument and `a` as the second argument, and initializes `comp` with `compare`; calls `c.insert(c.end(), first, last)`; and finally calls `make_heap(c.begin(), c.end(), comp)`.

```
template<class InputIterator, class Alloc>
  priority_queue(InputIterator first, InputIterator last, const Compare& compare, Container&& cont, const Alloc& a);
```

-?- *Effects:* Initializes `c` with `std::move(cont)` as the first argument and `a` as the second argument, and initializes `comp` with `compare`; calls `c.insert(c.end(), first, last)`; and finally calls `make_heap(c.begin(), c.end(), comp)`.

3517. `join_view::iterator`'s `iter_swap` is underconstrained

Section: 24.7.11.3 [range.join.iterator] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2021-01-28 **Last modified:** 2021-01-31

Priority: Not Prioritized

View other active issues in [range.join.iterator].

View all other issues in [range.join.iterator].

Discussion:

`std::ranges::join_view::iterator`'s hidden friend `iter_swap` is specified in 24.7.11.3 [range.join.iterator]/16 as:

```
friend constexpr void iter_swap(const iterator& x, const iterator& y)
  noexcept(noexcept(ranges::iter_swap(x.inner_, y.inner_)));
```

-16- *Effects:* Equivalent to: `return ranges::iter_swap(x.inner_, y.inner_);`

Notably, the expression `ranges::iter_swap(meow, woof)` is not valid for all iterators `meow` and `woof`, or even all input iterators of the same type as is the case here. This `iter_swap` overload should be constrained to require the type of `iterator::inner_` (`iterator_t<range_reference_t<maybe-const<Const, V>>>`) to satisfy `indirectly_swappable`. Notably this is already the case for `iter_swap` friends of every other iterator adaptor in the Standard Library (`reverse_iterator`, `move_iterator`, `common_iterator`, `counted_iterator`, `filter_view::iterator`, `transform_view::iterator`, and `split_view::inner-iterator`). The omission for `join_view::iterator` seems to have simply been an oversight.

[2021-03-12; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Modify 24.7.11.3 [range.join.iterator] as indicated:

> [*Drafting note:* If 3500 is accepted before this issue, it is kindly suggested to the Project Editor to apply the equivalent replacement of

"`iterator_t<range_reference_t<Base>>`" by "*`InnerIter`*" to the newly inserted `requires`.

```
namespace std::ranges {
  template<input_range V>
    requires view<V> && input_range<range_reference_t<V>> &&
             (is_reference_v<range_reference_t<V>> ||
             view<range_value_t<V>>)
  template<bool Const>
  struct join_view<V>::iterator {
    […]

    friend constexpr void iter_swap(const iterator& x, const iterator& y)
      noexcept(noexcept(ranges::iter_swap(x.inner_, y.inner_)))
      requires indirectly_swappable<iterator_t<range_reference_t<Base>>>;
  };
}
```

[…]

```
friend constexpr void iter_swap(const iterator& x, const iterator& y)
  noexcept(noexcept(ranges::iter_swap(x.inner_, y.inner_)))
  requires indirectly_swappable<iterator_t<range_reference_t<Base>>>;
```

-16- *Effects:* Equivalent to: `return ranges::iter_swap(x.inner_, y.inner_);`

3518. Exception requirements on char trait operations unclear

Section: 21.2.2 [char.traits.require] **Status:** Tentatively Ready **Submitter:** Zoe Carver **Opened:** 2021-02-01 **Last modified:** 2021-02-02

Priority: Not Prioritized

View other active issues in [char.traits.require].

View all other issues in [char.traits.require].

Discussion:

21.2.2 [char.traits.require] p1 says:

> `X` denotes a traits class defining types and functions for the character container type `C` […] Operations on `X` shall not throw exceptions.

It should be clarified what "operations on `X`" means. For example, in this patch, there was some confusion around the exact meaning of "operations on `X`". If it refers to the expressions specified in [tab:char.traits.req] or if it refers to all member functions of `X`, this should be worded in some clearer way.

[2021-03-12; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Modify 21.2.2 [char.traits.require] as indicated:

> -1- In Table [tab:char.traits.req], `X` denotes a traits class defining types and functions for the character container type `C`; […] ~~Operations on `X` shall not throw exceptions~~No expression which is part of the character traits requirements specified in this subclause 21.2.2 [char.traits.require] shall exit via an exception.

3519. Incomplete synopses for `<random>` classes

Section: 26.6.4 [rand.eng], 26.6.5 [rand.adapt], 26.6.9 [rand.dist] **Status:** Tentatively Ready **Submitter:** Jens Maurer **Opened:** 2021-02-02 **Last modified:** 2021-04-19

Priority: 3

View all other issues in [rand.eng].

Discussion:

The synopses for the engine and distribution classes in `<random>` do not show declarations `operator==`, `operator!=`, `operator<<`, and `operator>>`, although they are part of the engine and distribution requirements.

Suggested resolution:

Add these operators as hidden friends to the respective class synopses.

[2021-02-07: Daniel provides concrete wording]

The proposed wording attempts to use a conservative approach in regard to exception specifications. Albeit 26.6.4.1 [rand.eng.general] p3 says that "no function described in 26.6.4 [rand.eng] throws an exception" (unless specified otherwise), not all implementations have marked the equality operators as `noexcept` (But some do for their engines, such as VS 2019). [No implementations marks the IO stream operators as `noexcept` of-course, because these cannot be prevented to throw exceptions in general by design].

The wording also uses hidden friends of the IO streaming operators ("inserters and extractors") as suggested by the issue discussion, but it should be noted that at least some existing implementations use out-of-class free function templates for their distributions.

Note that we intentionally don't declare any `operator!=`, because the auto-generated form already has the right semantics. The wording also covers the engine adaptor class templates because similar requirements apply to them.

[2021-03-12; Reflector poll]

Set priority to 3 following reflector poll.

[2021-03-12; LWG telecon]

Set status to Tentatively Ready after discussion and poll.

F AN
100 0

Proposed resolution:

This wording is relative to N4878.

1. Modify 26.6.4.2 [rand.eng.lcong] as indicated:

```
template<class UIntType, UIntType a, UIntType c, UIntType m>
class linear_congruential_engine {
  […]
  // constructors and seeding functions
  […]

  // equality operators
  friend bool operator==(const linear_congruential_engine& x, const linear_congruential_engine& y);

  // generating functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const linear_congruential_engine& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, linear_congruential_engine& x);
};
```

2. Modify 26.6.4.3 [rand.eng.mers] as indicated:

```
template<class UIntType, size_t w, size_t n, size_t m, size_t r,
            UIntType a, size_t u, UIntType d, size_t s,
            UIntType b, size_t t,
            UIntType c, size_t l, UIntType f>
class mersenne_twister_engine {
  […]
  // constructors and seeding functions
  […]

  // equality operators
  friend bool operator==(const mersenne_twister_engine& x, const mersenne_twister_engine& y);

  // generating functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const mersenne_twister_engine& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, mersenne_twister_engine& x);
};
```

3. Modify 26.6.4.4 [rand.eng.sub] as indicated:

```
template<class UIntType, size_t w, size_t s, size_t r>
class subtract_with_carry_engine {
  […]
  // constructors and seeding functions
  […]

  // equality operators
  friend bool operator==(const subtract_with_carry_engine& x, const subtract_with_carry_engine& y);
```

```
  // generating functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const subtract_with_carry_engine& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, subtract_with_carry_engine& x);
};
```

4. Modify 26.6.5.2 [rand.adapt.disc] as indicated:

```
template<class Engine, size_t p, size_t r>
class discard_block_engine {
  […]
  // constructors and seeding functions
  […]

  // equality operators
  friend bool operator==(const discard_block_engine& x, const discard_block_engine& y);

  // generating functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const discard_block_engine& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, discard_block_engine& x);
};
```

5. Modify 26.6.5.3 [rand.adapt.ibits] as indicated:

```
template<class Engine, size_t w, class UIntType>
class independent_bits_engine {
  […]
  // constructors and seeding functions
  […]

  // equality operators
  friend bool operator==(const independent_bits_engine& x, const independent_bits_engine& y);

  // generating functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const independent_bits_engine& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, independent_bits_engine& x);
};
```

6. Modify 26.6.5.4 [rand.adapt.shuf] as indicated:

```
template<class Engine, size_t k>
class shuffle_order_engine {
  […]
  // constructors and seeding functions
  […]

  // equality operators
  friend bool operator==(const shuffle_order_engine& x, const shuffle_order_engine& y);

  // generating functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const shuffle_order_engine& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, shuffle_order_engine& x);
};
```

7. Modify 26.6.9.2.1 [rand.dist.uni.int] as indicated:

```
template<class IntType = int>
class uniform_int_distribution {
  […]
  // constructors and reset functions
  […]
```

```
  // equality operators
  friend bool operator==(const uniform_int_distribution& x, const uniform_int_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const uniform_int_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, uniform_int_distribution& x);
};
```

8. Modify 26.6.9.2.2 [rand.dist.uni.real] as indicated:

```
template<class RealType = double>
class uniform_real_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const uniform_real_distribution& x, const uniform_real_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const uniform_real_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, uniform_real_distribution& x);
};
```

9. Modify 26.6.9.3.1 [rand.dist.bern.bernoulli] as indicated:

```
class bernoulli_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const bernoulli_distribution& x, const bernoulli_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const bernoulli_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, bernoulli_distribution& x);
};
```

10. Modify 26.6.9.3.2 [rand.dist.bern.bin] as indicated:

```
template<class IntType = int>
class binomial_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const binomial_distribution& x, const binomial_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
```

```
    operator<<(basic_ostream<charT, traits>& os, const binomial_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
    operator>>(basic_istream<charT, traits>& is, binomial_distribution& x);
};
```

11. Modify 26.6.9.3.3 [rand.dist.bern.geo] as indicated:

```
template<class IntType = int>
class geometric_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const geometric_distribution& x, const geometric_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const geometric_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
    operator>>(basic_istream<charT, traits>& is, geometric_distribution& x);
};
```

12. Modify 26.6.9.3.4 [rand.dist.bern.negbin] as indicated:

```
template<class IntType = int>
class negative_binomial_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const negative_binomial_distribution& x, const negative_binomial_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const negative_binomial_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
    operator>>(basic_istream<charT, traits>& is, negative_binomial_distribution& x);
};
```

13. Modify 26.6.9.4.1 [rand.dist.pois.poisson] as indicated:

```
template<class IntType = int>
class poisson_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const poisson_distribution& x, const poisson_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const poisson_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
    operator>>(basic_istream<charT, traits>& is, poisson_distribution& x);
};
```

14. Modify 26.6.9.4.2 [rand.dist.pois.exp] as indicated:

```
template<class RealType = double>
class exponential_distribution {
  […]
```

```
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const exponential_distribution& x, const exponential_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const exponential_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, exponential_distribution& x);
};
```

15. Modify 26.6.9.4.3 [rand.dist.pois.gamma] as indicated:

```
template<class RealType = double>
class gamma_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const gamma_distribution& x, const gamma_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const gamma_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, gamma_distribution& x);
};
```

16. Modify 26.6.9.4.4 [rand.dist.pois.weibull] as indicated:

```
template<class RealType = double>
class weibull_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const weibull_distribution& x, const weibull_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const weibull_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, weibull_distribution& x);
};
```

17. Modify 26.6.9.4.5 [rand.dist.pois.extreme] as indicated:

```
template<class RealType = double>
class extreme_value_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const extreme_value_distribution& x, const extreme_value_distribution& y);

  // generating functions
  […]

  // property functions
  […]
```

```
  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const extreme_value_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, extreme_value_distribution& x);
};
```

18. Modify 26.6.9.5.1 [rand.dist.norm.normal] as indicated:

```
template<class RealType = double>
class normal_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const normal_distribution& x, const normal_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const normal_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, normal_distribution& x);
};
```

19. Modify 26.6.9.5.2 [rand.dist.norm.lognormal] as indicated:

```
template<class RealType = double>
class lognormal_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const lognormal_distribution& x, const lognormal_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const lognormal_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, lognormal_distribution& x);
};
```

20. Modify 26.6.9.5.3 [rand.dist.norm.chisq] as indicated:

```
template<class RealType = double>
class chi_squared_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const chi_squared_distribution& x, const chi_squared_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const chi_squared_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, chi_squared_distribution& x);
};
```

21. Modify 26.6.9.5.4 [rand.dist.norm.cauchy] as indicated:

```
template<class RealType = double>
class cauchy_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const cauchy_distribution& x, const cauchy_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const cauchy_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, cauchy_distribution& x);
};
```

22. Modify 26.6.9.5.5 [rand.dist.norm.f] as indicated:

```
template<class RealType = double>
class fisher_f_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const fisher_f_distribution& x, const fisher_f_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const fisher_f_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, fisher_f_distribution& x);
};
```

23. Modify 26.6.9.5.6 [rand.dist.norm.t] as indicated:

```
template<class RealType = double>
class student_t_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const student_t_distribution& x, const student_t_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const student_t_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, student_t_distribution& x);
};
```

24. Modify 26.6.9.6.1 [rand.dist.samp.discrete] as indicated:

```
template<class IntType = int>
class discrete_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const discrete_distribution& x, const discrete_distribution& y);

  // generating functions
  […]
```

```
  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const discrete_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, discrete_distribution& x);
};
```

25. Modify 26.6.9.6.2 [rand.dist.samp.pconst] as indicated:

```
template<class RealType = double>
class piecewise_constant_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const piecewise_constant_distribution& x, const piecewise_constant_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const piecewise_constant_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, piecewise_constant_distribution& x);
};
```

26. Modify 26.6.9.6.3 [rand.dist.samp.plinear] as indicated:

```
template<class RealType = double>
class piecewise_linear_distribution {
  […]
  // constructors and reset functions
  […]

  // equality operators
  friend bool operator==(const piecewise_linear_distribution& x, const piecewise_linear_distribution& y);

  // generating functions
  […]

  // property functions
  […]

  // inserters and extractors
  template<class charT, class traits>
    friend basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os, const piecewise_linear_distribution& x);
  template<class charT, class traits>
    friend basic_istream<charT, traits>&
      operator>>(basic_istream<charT, traits>& is, piecewise_linear_distribution& x);
};
```

3520. `iter_move` and `iter_swap` are inconsistent for `transform_view::iterator`

Section: 24.7.6.3 [range.transform.iterator] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-02-03 **Last modified:** 2021-04-19

Priority: 2

View other active issues in [range.transform.iterator].

View all other issues in [range.transform.iterator].

Discussion:

For `transform_view::iterator`, `iter_move` is specified to operate on the transformed value but `iter_swap` is specified to operate on the underlying iterator.

Consider the following test case:

```
struct X { int x; int y; };
std::vector<X> v = {...};
auto t = v | views::transform(&X::x);
ranges::sort(t);
```

`iter_swap` on `t`'s iterators would swap the whole `x`, including the `y` part, but `iter_move` will only move the `x` data member and leave the `y` part intact. Meanwhile, `ranges::sort` can use both `iter_move` and `iter_swap`, and does so in at least one implementation. The mixed behavior means that we get neither "sort `X`s by their `x` data member" (as `ranges::sort(v, {}, &X::x)` would do), nor "sort the `x` data member of these `X`s and leave the rest unchanged", as one might expect, but instead some arbitrary permutation of `y`. This seems like a questionable state of affairs.

[2021-03-12; Reflector poll]

Set priority to 2 following reflector poll.

[2021-03-12; LWG telecon]

Set status to Tentatively Ready after discussion and poll.

F A N
9 0 0

Proposed resolution:

This wording is relative to N4878.

1. Modify 24.7.6.3 [range.transform.iterator] as indicated:

```
namespace std::ranges {
  template<input_range V, copy_constructible F>
    requires view<V> && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<V>> &&
             can-reference<invoke_result_t<F&, range_reference_t<V>>>
  template<bool Const>
  class transform_view<V, F>::iterator {
    […]
    friend constexpr void iter_swap(const iterator& x, const iterator& y)
      noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
      requires indirectly_swappable<iterator_t<Base>>;
  };
}
```

[…]

~~`friend constexpr void iter_swap(const iterator& x, const iterator& y) noexcept(noexcept(ranges::iter_swap(x.current_, y.current_))) requires indirectly_swappable<iterator_t<Base>>;`~~

~~23 *Effects:* Equivalent to `ranges::iter_swap(x.current_, y.current_);`~~

3521. Overly strict requirements on `qsort` and `bsearch`

Section: 25.12 [alg.c.library] **Status:** Tentatively Ready **Submitter:** Richard Smith **Opened:** 2021-02-02 **Last modified:** 2021-02-23

Priority: Not Prioritized

View all other issues in [alg.c.library].

Discussion:

Per 25.12 [alg.c.library]/2, for `qsort` and `bsearch`, we have:

> *Preconditions:* The objects in the array pointed to by `base` are of trivial type.

This seems like an unnecessarily strict requirement. `qsort` only needs the objects to be of a trivially-copyable type (because it will use `memcpy` or equivalent to relocate them), and `bsearch` doesn't need any particular properties of the array element type. Presumably it would be in improvement to specify the more-precise requirements instead.

We should also reconsider the other uses of the notion of a trivial type. It's really not a useful or meaningful type property by itself, because it doesn't actually require that any operations on the type are valid (due to the possibility of them being ambiguous or only some of them being available) and the places that consider it very likely actually mean `is_trivially_copyable` plus `is_trivially_default_constructible` instead, or perhaps `is_trivially_copy_constructible` and `is_trivially_move_constructible` and so on.

Other than `qsort` and `bsearch`, the only uses of this type property in the standard are to constrain `max_align_t`, `aligned_storage`, `aligned_union`, and the element type of `basic_string` (and in the definition of the deprecated `is_pod` trait), all of which (other than `is_pod`) I think really mean "is trivially default constructible", not "has at least one eligible default constructor and all eligible default constructors are trivial". And in fact I think the alignment types are underspecified — we don't want to require merely that they be trivially-copyable, since that doesn't require any particular operation on them to actually be valid — we also want to require that they actually model `semiregular`.

[2021-02-23; Casey Carter provides concrete wording]

[2021-03-12; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Modify 25.12 [alg.c.library] as indicated:

```
void* bsearch(const void* key, const void* base, size_t nmemb, size_t size,
              c-compare-pred* compar);
void* bsearch(const void* key, const void* base, size_t nmemb, size_t size,
               compare-pred* compar);
void qsort(void* base, size_t nmemb, size_t size, c-compare-pred* compar);
void qsort(void* base, size_t nmemb, size_t size, compare-pred* compar);
```

-2- *Preconditions:* For `qsort`, t~~T~~he objects in the array pointed to by *base* are of ~~trivial~~trivially copyable type.

3522. Missing requirement on `InputIterator` template parameter for `priority_queue` constructors

Section: 22.6.5 [priority.queue] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-02-17 **Last modified:** 2021-02-26

Priority: Not Prioritized

View other active issues in [priority.queue].

View all other issues in [priority.queue].

Discussion:

There is nothing in 22.6.5 [priority.queue] or more generally 22.6 [container.adaptors] saying that `InputIterator` in the following constructor templates has to be an input iterator.

```
template<class InputIterator>
    priority_queue(InputIterator first, InputIterator last, const Compare& x,
                   const Container&);
template<class InputIterator>
    priority_queue(InputIterator first, InputIterator last,
                   const Compare& x = Compare(), Container&& = Container());
```

The second constructor template above therefore accepts

```
std::priority_queue<int> x = {1, 2};
```

to produce a `priority_queue` that contains a single element `2`. This behavior seems extremely questionable.

[2021-02-26; LWG telecon]

Set status to Tentatively Ready after discussion and poll.

F A N
11 0 0

Proposed resolution:

> [*Drafting note*: Because an upcoming paper provides iterator-pair constructors for other container adaptors, the wording below adds the restriction to 22.6.1 [container.adaptors.general] so that it also covers the constructors that will be added by that paper. — *end drafting note*]

This wording is relative to N4878.

1. Add the following paragraph to 22.6.1 [container.adaptors.general] after p3:

> -?- A constructor template of a container adaptor shall not participate in overload resolution if it has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.
>
> -4- A deduction guide for a container adaptor shall not participate in overload resolution if any of the following are true:
>
> (4.1) — It has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.
>
> […]

3523. `iota_view::sentinel` is not always `iota_view`'s sentinel

Section: 24.6.4.2 [range.iota.view] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-02-17 **Last modified:** 2021-02-18

Priority: Not Prioritized

View all other issues in [range.iota.view].

Discussion:

P1739R4 added the following constructor to `iota_view`:

```
constexpr iota_view(iterator first, sentinel last) : iota_view(*first, last.bound_) {}
```

However, while `iota_view`'s iterator type is always `iota_view::iterator`, its sentinel type is not always `iota_view::sentinel`. First, if `Bound` is `unreachable_sentinel_t`, then the sentinel type is `unreachable_sentinel_t` too - we don't add an unnecessary level of wrapping on top. Second, when `W` and `Bound` are the same type, `iota_view` models `common_range`, and the sentinel type is the same as the iterator type - that is, *`iterator`*, not *`sentinel`*.

Presumably the intent is to use the view's actual sentinel type, rather than always use the *`sentinel`* type.

[2021-03-12; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Edit 24.6.4.2 [range.iota.view], as indicated:

```
namespace std::ranges {
  // [...]

  template<weakly_incrementable W, semiregular Bound = unreachable_sentinel_t>
    requires weakly-equality-comparable-with<W, Bound> && semiregular<W>
  class iota_view : public view_interface<iota_view<W, Bound>> {
  private:
    // [range.iota.iterator], class iota_view::iterator
    struct iterator;            // exposition only
    // [range.iota.sentinel], class iota_view::sentinel
    struct sentinel;            // exposition only
    W value_ = W();             // exposition only
    Bound bound_ = Bound();     // exposition only
  public:
    iota_view() = default;
    constexpr explicit iota_view(W value);
    constexpr iota_view(type_identity_t<W> value,
                        type_identity_t<Bound> bound);
    constexpr iota_view(iterator first, sentinelsee below last);: iota_view(*first, last.bound_) {}

    constexpr iterator begin() const;
    constexpr auto end() const;
    constexpr iterator end() const requires same_as<W, Bound>;

    constexpr auto size() const requires see below;
  };

  template<class W, class Bound>
      requires (!is-integer-like<W> || !is-integer-like<Bound> ||
              (is-signed-integer-like<W> == is-signed-integer-like<Bound>))
      iota_view(W, Bound) -> iota_view<W, Bound>;
}

[...]

constexpr iota_view(type_identity_t<W> value, type_identity_t<Bound> bound);
```

-8- *Preconditions*: `Bound` denotes `unreachable_sentinel_t` or `bound` is reachable from `value`. When `W` and `Bound` model `totally_ordered_with`, then `bool(value <= bound)` is true.

-9- *Effects*: Initializes *`value_`* with `value` and *`bound_`* with `bound`.

`constexpr iota_view(iterator first, see below last);`

-?- *Effects*: Equivalent to:

(?.1) — If `same_as<W, Bound>` is `true`, `iota_view(first.value_, last.value_)`.

(?.2) — Otherwise, if `Bound` denotes `unreachable_sentinel_t`, `iota_view(first.value_, last)`.

(?.3) — Otherwise, `iota_view(first.value_, last.bound_)`.

-?- *Remarks*: The type of `last` is:

(?.1) — If `same_as<W, Bound>` is `true`, *`iterator`*.

(?.2) — Otherwise, if `Bound` denotes `unreachable_sentinel_t`, `Bound`.

(?.3) — Otherwise, *`sentinel`*.

3526. Return types of `uses_allocator_construction_args` unspecified

Section: 20.10.8.2 [allocator.uses.construction] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2021-02-25 **Last modified:** 2021-05-26

Priority: 3

View other active issues in [allocator.uses.construction].

View all other issues in [allocator.uses.construction].

Discussion:

The synopsis of `<memory>` in 20.10.2 [memory.syn] declares six overloads of `uses_allocator_construction_args` with return types "*see below*":

```
template<class T, class Alloc, class... Args>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                  Args&&... args) noexcept -> see below;
template<class T, class Alloc, class Tuple1, class Tuple2>>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc, piecewise_construct_t,
                                                  Tuple1&& x, Tuple2&& y)
                                                  noexcept -> see below;
template<class T, class Alloc>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc) noexcept -> see below;
template<class T, class Alloc, class U, class V>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                  U&& u, V&& v) noexcept -> see below;
template<class T, class Alloc, class U, class V>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                  const pair<U, V>& pr) noexcept -> see below;
template<class T, class Alloc, class U, class V>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                  pair<U, V>&& pr) noexcept -> see below;
```

The "*see below*s" also appear in the detailed specification of these overloaded function templates in 20.10.8.2 [allocator.uses.construction] para 4 through 15. Despite that the values these function templates return are specified therein, the return types are not. Presumably LWG wanted to specify deduced return types, but couldn't figure out how to do so, and just gave up?

[2021-02-27; Daniel comments and provides wording]

My interpretation is that the appearance of the *trailing-return-type* was actually unintended and that these functions where supposed to use the return type placeholder to signal the intention that the actual return type is deduced by the consistent sum of all return statements as they appear in the prototype specifications. Given that at least one implementation has indeed realized this form, I suggest to simply adjust the specification to remove the *trailing-return-type*. Specification-wise we have already existing practice for this approach (See e.g. `to_address`).

[2021-03-12; Reflector poll]

Set priority to 3 following reflector poll.

[2021-05-26; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Edit 20.10.2 [memory.syn], header `<memory>` synopsis, as indicated:

```
namespace std {
  […]
  // 20.10.8.2 [allocator.uses.construction], uses-allocator construction
  template<class T, class Alloc, class... Args>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    Args&&... args) noexcept -> see below;
  template<class T, class Alloc, class Tuple1, class Tuple2>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, piecewise_construct_t,
                                                    Tuple1&& x, Tuple2&& y)
                                                    noexcept -> see below;
  template<class T, class Alloc>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc) noexcept -> see below;
  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    U&& u, V&& v) noexcept -> see below;
  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    const pair<U, V>& pr) noexcept -> see below;
  template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    pair<U, V>&& pr) noexcept -> see below;
  […]
}
```

2. Edit 20.10.8.2 [allocator.uses.construction] as indicated:

```
template<class T, class Alloc, class... Args>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                  Args&&... args) noexcept -> see below;
```

[…]

-5- *Returns:* A `tuple` value determined as follows:

(5.1) — if […], return `forward_as_tuple(std::forward<Args>(args)...)`.

(5.2) — Otherwise, if […], return

```
tuple<allocator_arg_t, const Alloc&, Args&&...>(
  allocator_arg, alloc, std::forward<Args>(args)...)
```

(5.3) — Otherwise, if […], return `forward_as_tuple(std::forward<Args>(args)..., alloc)`.

(5.4) — Otherwise, the program is ill-formed.

```
template<class T, class Alloc, class Tuple1, class Tuple2>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc, piecewise_construct_t,
                                                  Tuple1&& x, Tuple2&& y)
                                                  noexcept -> see below;
```

[…]

-7- *Effects:* For `T` specified as `pair<T1, T2>`, equivalent to:

```
return make_tuple(
  piecewise_construct,
  apply([&alloc](auto&&... args1) {
          return uses_allocator_construction_args<T1>(
            alloc, std::forward<decltype(args1)>(args1)...);
        }, std::forward<Tuple1>(x)),
  apply([&alloc](auto&&... args2) {
          return uses_allocator_construction_args<T2>(
            alloc, std::forward<decltype(args2)>(args2)...);
        }, std::forward<Tuple2>(y)));
```

```
template<class T, class Alloc>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc) noexcept -> see below;
```

[…]

-9- *Effects:* Equivalent to:

```
return uses_allocator_construction_args<T>(alloc, piecewise_construct,
                                           tuple<>{}, tuple<>{});
```

```
template<class T, class Alloc, class U, class V>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                  U&& u, V&& v) noexcept -> see below;
```

[…]

-11- *Effects:* Equivalent to:

```
return uses_allocator_construction_args<T>(alloc, piecewise_construct,
                                           forward_as_tuple(std::forward<U>(u)),
                                           forward_as_tuple(std::forward<V>(v));
```

```
template<class T, class Alloc, class U, class V>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                  const pair<U, V>& pr) noexcept -> see below;
```

[…]

-13- *Effects:* Equivalent to:

```
return uses_allocator_construction_args<T>(alloc, piecewise_construct,
                                           forward_as_tuple(pr.first),
                                           forward_as_tuple(pr.second));
```

```
template<class T, class Alloc, class U, class V>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                  pair<U, V>&& pr) noexcept -> see below;
```

[…]

-15- *Effects:* Equivalent to:

```
return uses_allocator_construction_args<T>(alloc, piecewise_construct,
                                           forward_as_tuple(std::move(pr).first),
                                           forward_as_tuple(std::move(pr).second));
```

3527. `uses_allocator_construction_args` handles rvalue pairs of rvalue references incorrectly

Section: 20.10.8.2 [allocator.uses.construction] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-02-27 **Last modified:** 2021-02-27

Priority: Not Prioritized

View other active issues in [allocator.uses.construction].

View all other issues in [allocator.uses.construction].

Discussion:

For an rvalue pair `pr`, `uses_allocator_construction_args` is specified to forward `std::move(pr).first` and `std::move(pr).second`. This is correct for non-references and lvalue references, but wrong for rvalue refrences because the class member access produces an lvalue (see 7.6.1.5 [expr.ref]/6). `get` produces an xvalue, which is what is desired here.

[2021-03-12; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Edit 20.10.8.2 [allocator.uses.construction] as indicated:

```
template<class T, class Alloc, class U, class V>
  constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                  pair<U, V>&& pr) noexcept -> see below;
```

[…]

-15- *Effects:* Equivalent to:

```
return uses_allocator_construction_args<T>(alloc, piecewise_construct,
                                           forward_as_tuple(~~std::move(pr).first~~get<0>(std::move(pr))),
                                           forward_as_tuple(~~std::move(pr).second~~get<1>(std::move(pr))));
```

3528. `make_from_tuple` can perform (the equivalent of) a C-style cast

Section: 20.5.5 [tuple.apply] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-02-28 **Last modified:** 2021-03-12

Priority: 3

Discussion:

`make_from_tuple` is specified to return `T(get<I>(std::forward<Tuple>(t))...)`. When there is only a single tuple element, this is equivalent to a C-style cast that may be a `reinterpret_cast`, a `const_cast`, or an access-bypassing `static_cast`.

[2021-03-12; Reflector poll]

Set priority to 3 following reflector poll. Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Edit 20.5.5 [tuple.apply] as indicated:

```
template<class T, class Tuple>
  constexpr T make_from_tuple(Tuple&& t);
```

[…]

-2- *Effects:* Given the exposition-only function:

```
template<class T, class Tuple, size_t... I>
  requires is_constructible_v<T, decltype(get<I>(declval<Tuple>()))...>
constexpr T make-from-tuple-impl(Tuple&& t, index_sequence<I...>) {     // exposition only
  return T(get<I>(std::forward<Tuple>(t))...);
}
```

Equivalent to:

```
return make-from-tuple-impl<T>(
  std::forward<Tuple>(t),
  make_index_sequence<tuple_size_v<remove_reference_t<Tuple>>>{});
```

[*Note 1:* The type of T must be supplied as an explicit template parameter, as it cannot be deduced from the argument list. — *end note*]

3529. `priority_queue(first, last)` should construct `c` with `(first, last)`

Section: 22.6.5 [priority.queue] **Status:** Tentatively Ready **Submitter:** Arthur O'Dwyer **Opened:** 2021-03-01 **Last modified:** 2021-03-05

Priority: Not Prioritized

View other active issues in [priority.queue].

View all other issues in [priority.queue].

Discussion:

Tim's new constructors for `priority_queue` (LWG 3506) are specified so that when you construct

```
auto pq = PQ(first, last, a);
```

it calls this new-in-LWG3506 constructor:

```
template<class InputIterator, class Alloc>
  priority_queue(InputIterator first, InputIterator last, const Alloc& a);
```

Effects: Initializes `c` with `first` as the first argument, `last` as the second argument, and `a` as the third argument, and value-initializes `comp`; calls `make_heap(c.begin(), c.end(), comp)`.

But the pre-existing constructors are specified so that when you construct

```
auto pq = PQ(first, last);
```

it calls this pre-existing constructor:

```
template<class InputIterator>
  priority_queue(InputIterator first, InputIterator last, const Compare& x = Compare(), Container&& y = Container());
```

Preconditions: `x` defines a strict weak ordering ([alg.sorting]).

Effects: Initializes `comp` with `x` and `c` with `y` (copy constructing or move constructing as appropriate); calls `c.insert(c.end(), first, last)`; and finally calls `make_heap(c.begin(), c.end(), comp)`.

In other words,

```
auto pq = PQ(first, last);
```

will default-construct a `Container`, then move-construct `c` from that object, then `c.insert(first, last)`, and finally `make_heap`.

But our new

```
auto pq = PQ(first, last, a);
```

will simply construct `c` with `(first, last)`, then `make_heap`.

The latter is obviously better.

Also, Corentin's P1425R3 specifies the new iterator-pair constructors for `stack` and `queue` to construct `c` from `(first, last)`. Good.

LWG should refactor the existing constructor overload set so that the existing non-allocator-taking constructors simply construct `c` from `(first, last)`. This will improve consistency with the resolutions of LWG3506 and P1425, and reduce the surprise factor for users.

[2021-03-12; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Edit 22.6.5.1 [priqueue.overview] as indicated:

```
template<class InputIterator>
    priority_queue(InputIterator first, InputIterator last, const Compare& x = Compare());
template<class InputIterator>
    priority_queue(InputIterator first, InputIterator last, const Compare& x, const Container& y);
template<class InputIterator>
    priority_queue(InputIterator first, InputIterator last, const Compare& x~~ = Compare()~~, Container&& y~~ = Container()~~);
```

2. Edit 22.6.5.2 [priqueue.cons] as indicated:

```
template<class InputIterator>
    priority_queue(InputIterator first, InputIterator last, const Compare& x = Compare());
```

Preconditions: x defines a strict weak ordering ([alg.sorting]).

Effects: Initializes c with first as the first argument and last as the second argument, and initializes comp with x; then calls make_heap(c.begin(), c.end(), comp).

```
template<class InputIterator>
    priority_queue(InputIterator first, InputIterator last, const Compare& x, const Container& y);
template<class InputIterator>
    priority_queue(InputIterator first, InputIterator last, const Compare& x = Compare(), Container&& y = Container());
```

Preconditions: x defines a strict weak ordering ([alg.sorting]).

Effects: Initializes comp with x and c with y (copy constructing or move constructing as appropriate); calls c.insert(c.end(), first, last); and finally calls make_heap(c.begin(), c.end(), comp).

3530. *BUILTIN-PTR-MEOW* should not opt the type out of syntactic checks

Section: 20.14.8.8 [comparisons.three.way], 20.14.9 [range.cmp] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-03-04 **Last modified:** 2021-03-08

Priority: Not Prioritized

Discussion:

The use of *BUILTIN-PTR-MEOW* for the constrained comparison function objects was needed to disable the semantic requirements on the associated concepts when the comparison resolves to a built-in operator comparing pointers: the comparison object is adding special handling for this case to produce a total order despite the core language saying otherwise, so requiring the built-in operator to then produce a total order as part of the semantic requirements doesn't make sense.

However, because it is specified as a disjunction on the constraint, it means that the comparison function objects are now required to accept types that don't even meet the syntactic requirements of the associated concept. For example, ranges::less requires all six comparison operators (because of totally_ordered_with) to be present … except when operator< on the arguments resolves to a built-in operator comparing pointers, in which case it just requires operator< and operator== (except that the latter isn't even required to be checked — it comes from the use of ranges::equal_to in the precondition of ranges::less). This seems entirely arbitrary.

[2021-03-12; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Edit 20.14.8.8 [comparisons.three.way] as indicated:

~~-1- In this subclause, *BUILTIN-PTR-THREE-WAY*(T, U) for types T and U is a boolean constant expression. *BUILTIN-PTR-THREE-WAY*(T, U) is true if and only if <=> in the expression~~

~~declval<T>() <=> declval<U>()~~

~~resolves to a built-in operator comparing pointers.~~

```
struct compare_three_way {
  template<class T, class U>
    requires three_way_comparable_with<T, U> || BUILTIN_PTR_THREE_WAY(T, U)
  constexpr auto operator()(T&& t, U&& u) const;

  using is_transparent = unspecified;
};

template<class T, class U>
  requires three_way_comparable_with<T, U> || BUILTIN_PTR_THREE_WAY(T, U)
constexpr auto operator()(T&& t, U&& u) const;
```

-?- *Constraints:* T and U satisfy three_way_comparable_with.

-2- *Preconditions:* If the expression std::forward<T>(t) <=> std::forward<U>(u) results in a call to a built-in operator <=> comparing pointers of type P, the conversion sequences from both T and U to P are equality-preserving (18.2 [concepts.equality]); otherwise, T and U model three_way_comparable_with.

-3- *Effects:*

(3.1) — If the expression std::forward<T>(t) <=> std::forward<U>(u) results in a call to a built-in operator <=> comparing pointers of type P, returns strong_ordering::less if (the converted value of) t precedes u in the implementation-defined strict total order over pointers (3.27 [defns.order.ptr]), strong_ordering::greater if u precedes t, and otherwise strong_ordering::equal.

(3.2) — Otherwise, equivalent to: return std::forward<T>(t) <=> std::forward<U>(u);

2. Edit 20.14.9 [range.cmp] as indicated:

~~-1- In this subclause, *BUILTIN-PTR-CMP*(T, *op*, U) for types T and U and where *op* is an equality (7.6.10 [expr.eq]) or relational operator (7.6.9 [expr.rel]) is a boolean constant expression. *BUILTIN-PTR-CMP*(T, *op*, U) is true if and only if *op* in the expression declval<T>() *op* declval<U>() resolves to a built-in operator comparing pointers.~~

```
struct ranges::equal_to {
  template<class T, class U>
    requires equality_comparable_with<T, U> || BUILTIN_PTR_CMP(T, ==, U)
  constexpr bool operator()(T&& t, U&& u) const;

  using is_transparent = unspecified;
};

template<class T, class U>
  requires equality_comparable_with<T, U> || BUILTIN_PTR_CMP(T, ==, U)
constexpr bool operator()(T&& t, U&& u) const;
```

-?- *Constraints:* T and U satisfy equality_comparable_with.

-2- *Preconditions:* If the expression `std::forward<T>(t) == std::forward<U>(u)` results in a call to a built-in operator `==` comparing pointers of type `P`, the conversion sequences from both `T` and `U` to `P` are equality-preserving (18.2 [concepts.equality]); otherwise, T and U model equality_comparable_with.

-3- *Effects:*

(3.1) — If the expression `std::forward<T>(t) == std::forward<U>(u)` results in a call to a built-in operator `==` comparing pointers of type `P`, returns `false` if either (the converted value of) `t` precedes `u` or `u` precedes `t` in the implementation-defined strict total order over pointers (3.27 [defns.order.ptr]) and otherwise `true`.

(3.2) — Otherwise, equivalent to: `return std::forward<T>(t) == std::forward<U>(u);`

```
struct ranges::not_equal_to {
    template<class T, class U>
      requires equality_comparable_with<T, U> || BUILTIN_PTR_CMP(T, ==, U)
    constexpr bool operator()(T&& t, U&& u) const;

    using is_transparent = unspecified;
};

template<class T, class U>
constexpr bool operator()(T&& t, U&& u) const;
```

-?- *Constraints:* T and U satisfy equality_comparable_with.

-4- ~~operator() has effects e~~*Effects:* Equivalent to:

```
return !ranges::equal_to{}(std::forward<T>(t), std::forward<U>(u));
```

```
struct ranges::greater {
  template<class T, class U>
    requires totally_ordered_with<T, U> || BUILTIN_PTR_CMP(T, <, U)
  constexpr bool operator()(T&& t, U&& u) const;

  using is_transparent = unspecified;
};

template<class T, class U>
constexpr bool operator()(T&& t, U&& u) const;
```

-?- *Constraints:* T and U satisfy totally_ordered_with.

-5- ~~operator() has effects e~~*Effects:* Equivalent to:

```
return ranges::less{}(std::forward<U>(u), std::forward<T>(t));
```

```
struct ranges::less {
  template<class T, class U>
    requires totally_ordered_with<T, U> || BUILTIN_PTR_CMP(T, <, U)
  constexpr bool operator()(T&& t, U&& u) const;

  using is_transparent = unspecified;
};

template<class T, class U>
  requires totally_ordered_with<T, U> || BUILTIN_PTR_CMP(T, <, U)
constexpr bool operator()(T&& t, U&& u) const;
```

-?- *Constraints:* T and U satisfy totally_ordered_with.

-6- *Preconditions:* If the expression `std::forward<T>(t) < std::forward<U>(u)` results in a call to a built-in operator `<` comparing pointers of type `P`, the conversion sequences from both `T` and `U` to `P` are equality-preserving (18.2 [concepts.equality]); otherwise, T and U model totally_ordered_with. For any expressions `ET` and `EU` such that `decltype((ET))` is `T` and `decltype((EU))` is `U`, exactly one of `ranges::less{}(ET, EU)`, `ranges::less{}(EU, ET)`, or `ranges::equal_to{}(ET, EU)` is `true`.

-7- *Effects:*

(7.1) — If the expression `std::forward<T>(t) < std::forward<U>(u)` results in a call to a built-in operator `<` comparing pointers of type `P`, returns `true` if (the converted value of) `t` precedes `u` in the implementation-defined strict total order over pointers (3.27 [defns.order.ptr]) and otherwise `false`.

(7.2) — Otherwise, equivalent to: `return std::forward<T>(t) < std::forward<U>(u);`

```
struct ranges::greater_equal {
  template<class T, class U>
    requires totally_ordered_with<T, U> || BUILTIN_PTR_CMP(T, <, U)
  constexpr bool operator()(T&& t, U&& u) const;

  using is_transparent = unspecified;
};

template<class T, class U>
constexpr bool operator()(T&& t, U&& u) const;
```

-?- *Constraints:* `T` and `U` satisfy `totally_ordered_with`.

-8- ~~`operator()` has effects e~~*Effects:* Equivalent to:

```
return !ranges::less{}(std::forward<T>(t), std::forward<U>(u));
```

```
struct ranges::less_equal {
  template<class T, class U>
    requires totally_ordered_with<T, U> || BUILTIN_PTR_CMP(T, <, U)
  constexpr bool operator()(T&& t, U&& u) const;

  using is_transparent = unspecified;
};

template<class T, class U>
constexpr bool operator()(T&& t, U&& u) const;
```

-?- *Constraints:* `T` and `U` satisfy `totally_ordered_with`.

-9- ~~`operator()` has effects e~~*Effects:* Equivalent to:

```
return !ranges::less{}(std::forward<U>(u), std::forward<T>(t));
```

3532. `split_view<V, P>::inner-iterator<true>::operator++(int)` should depend on `Base`

Section: 24.7.12.5 [range.split.inner] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2021-03-11 **Last modified:** 2021-04-20

Priority: Not Prioritized

Discussion:

`split_view<V, P>::inner-iterator<Const>::operator++(int)` is specified directly in the synopsis in 24.7.12.5 [range.split.inner] as:

```
constexpr decltype(auto) operator++(int) {
  if constexpr (forward_range<V>) {
    auto tmp = *this;
    ++*this;
    return tmp;
  } else
    ++*this;
}
```

The dependency on the properties of `V` here is odd given that we are wrapping an iterator obtained from a `maybe-const<Const, V>` (aka `Base`). It seems like this function should instead be concerned with `forward_range<Base>`.

[2021-04-20; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Modify 24.7.12.5 [range.split.inner] as indicated:

```
constexpr decltype(auto) operator++(int) {
  if constexpr (forward_range<~~V~~Base>) {
    auto tmp = *this;
    ++*this;
    return tmp;
  } else
    ++*this;
}
```

3533. Make `base() const &` consistent across iterator wrappers that supports `input_iterators`

Section: 24.7.5.3 [range.filter.iterator], 24.7.6.3 [range.transform.iterator], 24.7.16.3 [range.elements.iterator] **Status:** Tentatively Ready **Submitter:** Tomasz Kamiński **Opened:** 2021-03-14 **Last modified:** 2021-04-20

Priority: Not Prioritized

Discussion:

The resolution of LWG issue 3391 changed the `base()` function for the `counted_iterator`/`move_iterator` to return `const &`, because the previous specification prevented non-mutating uses of the base iterator (comparing against sentinel) and made `take_view` unimplementable. However, this change was not applied for all other iterators wrappers, that may wrap move-only input iterators. As consequence, we end-up with inconsistency where a user can perform the following operations on some adapters, but not on others (e. g. `take_view` uses `counted_iterator` so it supports them).

A. read the original value of the base iterator, by calling `operator*`

B. find position of an element in the underlying iterator, when sentinel is sized by calling `operator-` (e.g. all input iterators wrapped into `counted_iterator`).

To fix above, the proposed wording below proposes to modify the signature of `iterator::base() const &` member function for all iterators adapters that support input iterator. These include:

- `filter_view::`*`iterator`* (uses case B)
- `transform_view::`*`iterator`* (uses case A)
- `elements_view::`*`iterator`* (uses case B)
- `lazy_split_view<V, Pattern>::`*`inner-iterator`* (uses case B) if P2210 is accepted

Note: `common_iterator` does not expose the `base()` function (because it can either point to iterator or sentinel), so changes to above are not proposed. However, both (A) and (B) use cases are supported.

[2021-04-20; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

> If P2210 would become accepted, the corresponding subclause [range.lazy.split.inner] (?) for `lazy_split_view::`*`inner-iterator`* would require a similar change.

1. Modify 24.7.5.3 [range.filter.iterator] as indicated:

```
[…]
constexpr const iterator_t<V>& base() const &
  requires copyable<iterator_t<V>>;
[…]
```

[…]

```
constexpr const iterator_t<V>& base() const &
  requires copyable<iterator_t<V>>;
```

-5- *Effects:* Equivalent to: `return current_;`

2. Modify 24.7.6.3 [range.transform.iterator] as indicated:

```
[…]
constexpr const iterator_t<Base>& base() const &
  requires copyable<iterator_t<Base>>;
[…]
```

[…]

```
constexpr const iterator_t<Base>& base() const &
  requires copyable<iterator_t<Base>>;
```

-5- *Effects:* Equivalent to: `return current_;`

3. Modify 24.7.16.3 [range.elements.iterator] as indicated:

```
[…]
constexpr const iterator_t<Base>& base() const &
  requires copyable<iterator_t<Base>>;
[…]
```

[…]

```
constexpr const iterator_t<Base>& base() const &
  requires copyable<iterator_t<Base>>;
```

-3- *Effects:* Equivalent to: `return current_;`

3536. Should `chrono::from_stream()` assign zero to `duration` for failure?

Section: 27.5.11 [time.duration.io] **Status:** Tentatively Ready **Submitter:** Matt Stephanson **Opened:** 2021-03-19 **Last modified:** 2021-04-20

Priority: Not Prioritized

View all other issues in [time.duration.io].

Discussion:

The `duration` specialization of `from_stream` says in N4878 27.5.11 [time.duration.io]/3:

> If the parse parses everything specified by the parsing format flags without error, and yet none of the flags impacts a duration, `d` will be assigned a zero value.

This is in contrast to the other specializations that say, for example, 27.8.3.3 [time.cal.day.nonmembers]/8:

> If the parse fails to decode a valid day, `is.setstate(ios_base::failbit)` is called and `d` is not modified.

The wording ("none of the flags impacts a duration" vs. "parse fails to decode a valid [meow]") and semantics ("assigned a zero value" vs. "not modified") are different, and it's not clear why that should be so. It also leaves unspecified what should be done in case of a parse failure, for example parsing `"%j"` from a stream containing `"meow"`. 27.13 [time.parse]/12 says that `failbit` should be set, but neither it nor 27.5.11 [time.duration.io]/3 mention the `duration` result if parsing fails.

This has been discussed at the Microsoft STL project, where Howard Hinnant, coauthor of P0355R7 that added these functions, commented:

> This looks like a bug in the standard to me, due to two errors on my part.
>
> I believe that the standard should clearly say that if `failbit` is set, the parsed variable (`duration`, `time_point`, whatever) is not modified. I mistakenly believed that the definition of unformatted input function covered this behavior. But after review, I don't believe it does. Instead each extraction operator seems to say this separately.
>
> I also at first did not have my example implementation coded to leave the duration unchanged. So that's how the wording got in in the first place. Here's the commit where I fixed my implementation: HowardHinnant/date@d53db7a. And I failed to propagate that fix into the proposal/standard.

It would be clearer and simpler for users if the `from_stream` specializations were consistent in wording and behavior.

Thanks to Stephan T. Lavavej, Miya Natsuhara, and Howard Hinnant for valuable investigation and discussion of this issue.

[2021-04-20; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4878.

1. Modify 27.5.11 [time.duration.io] as indicated:

```
template<class charT, class traits, class Rep, class Period, class Alloc = allocator<charT>>
  basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                duration<Rep, Period>& d,
                basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);
```

-3- *Effects:* Attempts to parse the input stream `is` into the duration `d` using the format flags given in the NTCTS `fmt` as specified in 27.13 [time.parse]. ~~If the parse parses everything specified by the parsing format flags without error, and yet none of the flags impacts a duration, `d` will be assigned a zero value~~If the parse fails to decode a valid duration, `is.setstate(ios_base::failbit)` is called and `d` is not modified. If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null. If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

3539. `format_to` must not copy models of `output_iterator<const charT&>`

Section: 20.20.4 [format.functions] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2021-03-31 **Last modified:** 2021-04-20

Priority: Not Prioritized

View all other issues in [format.functions].

Discussion:

20.20.4 [format.functions] specifies the overloads of `format_to` as:

```
template<class Out, class... Args>
  Out format_to(Out out, string_view fmt, const Args&... args);
template<class Out, class... Args>
  Out format_to(Out out, wstring_view fmt, const Args&... args);
```

-8- *Effects:* Equivalent to:

```
using context = basic_format_context<Out, decltype(fmt)::value_type>;
return vformat_to(out, fmt, make_format_args<context>(args...));
```

```
template<class Out, class... Args>
  Out format_to(Out out, const locale& loc, string_view fmt, const Args&... args);
template<class Out, class... Args>
  Out format_to(Out out, const locale& loc, wstring_view fmt, const Args&... args);
  Out format_to(Out out, wstring_view fmt, const Args&... args);
```

-9- *Effects:* Equivalent to:

```
using context = basic_format_context<Out, decltype(fmt)::value_type>;
return vformat_to(out, loc, fmt, make_format_args<context>(args...));
```

but the overloads of `vformat_to` take their first argument by value (from the same subclause):

```
template<class Out>
  Out vformat_to(Out out, string_view fmt,
                 format_args_t<type_identity_t<Out>, char> args);
template<class Out>
  Out vformat_to(Out out, wstring_view fmt,
                 format_args_t<type_identity_t<Out>, wchar_t> args);
template<class Out>
  Out vformat_to(Out out, const locale& loc, string_view fmt,
                 format_args_t<type_identity_t<Out>, char> args);
template<class Out>
  Out vformat_to(Out out, const locale& loc, wstring_view fmt,
                 format_args_t<type_identity_t<Out>, wchar_t> args);
```

-10- Let `charT` be `decltype(fmt)::value_type`.

-11- *Constraints:* `Out` satisfies `output_iterator<const charT&>`.

-12- *Preconditions:* `Out` models `output_iterator<const charT&>`.

and require its type to model `output_iterator<const charT&>`. `output_iterator<T, U>` refines `input_or_output_iterator<T>` which refines `movable<T>`, but it notably does not refine `copyable<T>`. Consequently, the "Equivalent to" code for the `format_to` overloads is copying an iterator that could be move-only. I suspect it is not the intent that calls to `format_to` with move-only iterators be ill-formed, but that it was simply an oversight that this wording needs updating to be consistent with the change to allow move-only single-pass iterators in C++20.

[2021-04-20; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 20.20.4 [format.functions] as indicated:

```
template<class Out, class... Args>
  Out format_to(Out out, string_view fmt, const Args&... args);
template<class Out, class... Args>
  Out format_to(Out out, wstring_view fmt, const Args&... args);
```

-8- *Effects:* Equivalent to:

```
using context = basic_format_context<Out, decltype(fmt)::value_type>;
return vformat_to(std::move(out), fmt, make_format_args<context>(args...));
```

```
template<class Out, class... Args>
  Out format_to(Out out, const locale& loc, string_view fmt, const Args&... args);
template<class Out, class... Args>
  Out format_to(Out out, const locale& loc, wstring_view fmt, const Args&... args);
```

-9- *Effects:* Equivalent to:

```
using context = basic_format_context<Out, decltype(fmt)::value_type>;
return vformat_to(std::move(out), loc, fmt, make_format_args<context>(args...));
```

3540. §[format.arg] There should be no `const` in `basic_format_arg(const T* p)`

Section: 20.20.6.1 [format.arg] **Status:** Tentatively Ready **Submitter:** S. B. Tam **Opened:** 2021-04-07 **Last modified:** 2021-04-20

Priority: Not Prioritized

View other active issues in [format.arg].

View all other issues in [format.arg].

Discussion:

When P0645R10 "Text formatting" was merged into the draft standard, there was a typo: an exposition-only constructor of `basic_format_arg` is declared as accepting `const T*` in the class synopsis, but is later declared to accept `T*`. This was editorial issue 3461 and was resolved by adding `const` to the redeclaration.

As it is, constructing `basic_format_arg` from `void*` will select `template<class T> explicit basic_format_arg(const T& v)` and store a `basic_format_arg::handle` instead of select `template<class T> basic_format_arg(const T*)` and store a `const void*`, because `void*` → `void*const&` is identity conversion, while `void*` → `const void*` is qualification conversion.

While this technically works, it seems that storing a `const void*` would be more intuitive.

Hence, I think `const` should be removed from both declarations of `basic_format_arg(const T*)`, so that construction from `void*` will select this constructor, resulting in more intuitive behavior.

[2021-04-20; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 20.20.6.1 [format.arg] as indicated:

```
namespace std {
  template<class Context>
  class basic_format_arg {
  public:
    class handle;

  private:
    using char_type = typename Context::char_type;                    // exposition only

    variant<monostate, bool, char_type,
      int, unsigned int, long long int, unsigned long long int,
      float, double, long double,
      const char_type*, basic_string_view<char_type>,
      const void*, handle> value;                                     // exposition only

    template<class T> explicit basic_format_arg(const T& v) noexcept;  // exposition only
    explicit basic_format_arg(float n) noexcept;                      // exposition only
    explicit basic_format_arg(double n) noexcept;                     // exposition only
    explicit basic_format_arg(long double n) noexcept;                // exposition only
    explicit basic_format_arg(const char_type* s);                    // exposition only

    template<class traits>
      explicit basic_format_arg(
        basic_string_view<char_type, traits> s) noexcept;             // exposition only

    template<class traits, class Allocator>
      explicit basic_format_arg(
        const basic_string<char_type, traits, Allocator>& s) noexcept; // exposition only

    explicit basic_format_arg(nullptr_t) noexcept;                    // exposition only

    template<class T>
      explicit basic_format_arg(~~const~~ T* p) noexcept;             // exposition only
  public:
    basic_format_arg() noexcept;

    explicit operator bool() const noexcept;
  };
}
```

[…]

```
template<class T> explicit basic_format_arg(~~const~~ T* p) noexcept;
```

-12- *Constraints:* `is_void_v<T>` is `true`.

-13- *Effects:* Initializes `value` with `p`.

-14- [*Note 1*: Constructing `basic_format_arg` from a pointer to a member is ill-formed unless the user provides an enabled specialization of formatter for that pointer to member type. — *end note*]

3541. `indirectly_readable_traits` should be SFINAE-friendly for all types

Section: 23.3.2.2 [readable.traits] **Status:** Tentatively Ready **Submitter:** Christopher Di Bella **Opened:** 2021-04-08 **Last modified:** 2021-04-20

Priority: Not Prioritized

View all other issues in [readable.traits].

Discussion:

Following the outcome of LWG 3446, a strict implementation of `std::indirectly_readable_traits` isn't SFINAE-friendly for types that have different `value_type` and `element_type` members due to the ambiguity between the *`has-member-value-type`* and *`has-member-element-type`* specialisations. Other traits types are empty when requirements aren't met; we should follow suit here.

[2021-04-20; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 23.3.2.2 [readable.traits] p1 as indicated:

```
[…]
template<class T>
  concept has-member-value-type = requires { typename T::value_type; }; // exposition only

template<class T>
  concept has-member-element-type = requires { typename T::element_type; }; // exposition only

template<class> struct indirectly_readable_traits { };

[…]

template<has-member-value-type T>
struct indirectly_readable_traits<T>
  : cond-value-type<typename T::value_type> { };

template<has-member-element-type T>
struct indirectly_readable_traits<T>
  : cond-value-type<typename T::element_type> { };

template<has-member-value-type T>
  requires has-member-element-type<T>
struct indirectly_readable_traits<T> { };

template<has-member-value-type T>
  requires has-member-element-type<T> &&
           same_as<remove_cv_t<typename T::element_type>, remove_cv_t<typename T::value_type>>
struct indirectly_readable_traits<T>
  : cond-value-type<typename T::value_type> { };
[…]
```

3542. `basic_format_arg` mis-handles `basic_string_view` with custom traits

Section: 20.20.6.1 [format.arg] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2021-04-20 **Last modified:** 2021-05-10

Priority: Not Prioritized

View other active issues in [format.arg].

View all other issues in [format.arg].

Discussion:

`basic_format_arg` has a constructor that accepts a `basic_string_view` of an appropriate character type, with *any* `traits` type. The constructor is specified in 20.20.6.1 [format.arg] as:

```
template<class traits>
explicit basic_format_arg(basic_string_view<char_type, traits> s) noexcept;
```

-9- *Effects:* Initializes `value` with `s`.

Recall that `value` is a `variant<monostate, bool, char_type, int, unsigned int, long long int, unsigned long long int, float, double, long double, const char_type*, basic_string_view<char_type>, const void*, handle>` as specified earlier in the subclause. Since

`basic_string_view<meow, woof>` cannot be initialized with an lvalue `basic_string_view<meow, quack>` — and certainly none of the other alternative types can be initialized by such an lvalue — the effects of this constructor are ill-formed when `traits` is not `std::char_traits<char_type>`.

The `basic_string` constructor deals with this same issue by ignoring the deduced traits type when initializing `value`'s `basic_string_view` member:

```
template<class traits, class Allocator>
  explicit basic_format_arg(
    const basic_string<char_type, traits, Allocator>& s) noexcept;
```

-10- *Effects:* Initializes `value` with `basic_string_view<char_type>(s.data(), s.size())`.

which immediately begs the question of "why doesn't the `basic_string_view` constructor do the same?"

[2021-05-10; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 20.20.6.1 [format.arg] as indicated:

```
template<class traits>
  explicit basic_format_arg(basic_string_view<char_type, traits> s) noexcept;
```

-9- *Effects:* Initializes `value` with ~~s~~`basic_string_view<char_type>(s.data(), s.size())`.

3543. Definition of when `counted_iterators` refer to the same sequence isn't quite right

Section: 23.5.6.1 [counted.iterator] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-04-21 **Last modified:** 2021-05-10

Priority: Not Prioritized

View all other issues in [counted.iterator].

Discussion:

23.5.6.1 [counted.iterator]/3 says:

> Two values `i1` and `i2` of types `counted_iterator<I1>` and `counted_iterator<I2>` refer to elements of the same sequence if and only if `next(i1.base(), i1.count())` and `next(i2.base(), i2.count())` refer to the same (possibly past-the-end) element.

However, some users of `counted_iterator` (such as `take_view`) don't guarantee that there are `count()` elements past `base()`. It seems that we need to rephrase this definition to account for such uses.

[2021-05-10; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 23.5.6.1 [counted.iterator] as indicated:

> -3- Two values `i1` and `i2` of types `counted_iterator<I1>` and `counted_iterator<I2>` refer to elements of the same sequence if and only if there exists some integer *n* such that `next(i1.base(), i1.count()` `+ n`) and `next(i2.base(), i2.count()` `+ n`) refer to the same (possibly past-the-end) element.

3544. *`format-arg-store`*`::args` is unintentionally not exposition-only

Section: 20.20.6.2 [format.arg.store] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2021-04-22 **Last modified:** 2021-05-20

Priority: 3

Discussion:

Despite the statement in 20.20.6.3 [format.args]/1:

> An instance of `basic_format_args` provides access to formatting arguments. Implementations should optimize the representation of `basic_format_args` for a small number of formatting arguments. [*Note 1:* For example, by storing indices of type alternatives separately from values and packing the former. — *end note*]

`make_format_args` and `make_wformat_args` are specified to return an object whose type is a specialization of the exposition-only class template *`format-arg-store`* which has a public non-static data member that is an `array` of `basic_format_arg`. In order to actually "optimize the representation of `basic_format_args`" an implementation must internally avoid using `make_format_args` (and `make_wformat_args`) and instead use a different mechanism to

type-erase arguments. `basic_format_args` must still be convertible from *`format-arg-store`* as specified, however, so internally `basic_format_args` must support both the bad/slow standard mechanism and a good/fast internal-only mechanism for argument storage.

While this complicated state of affairs is technically implementable, it greatly complicates the implementation of `<format>` with no commensurate benefit. Indeed, naive users may make the mistake of thinking that e.g. `vformat(fmt, make_format_args(args...))` is as efficient as `format(fmt, args...)` — that's what the "*Effects:* Equivalent to" in 20.20.4 [format.functions]/2 implies — and inadvertently introduce performance regressions. It would be better for both implementers and users if *`format-arg-store`* had no public data members and its member `args` were made exposition-only.

[2021-05-10; Reflector poll]

Priority set to 3. Tim: "The current specification of `make_format_args` depends on *`format-arg-store`* being an aggregate, which is no longer true with this PR."

Previous resolution [SUPERSEDED]:

This wording is relative to N4885.

1. Modify 20.20.1 [format.syn], header `<format>` synopsis, as indicated:

```
[…]
// 20.20.6.2 [format.arg.store], class template format-arg-store
template<class Context, class... Args> structclass format-arg-store; // exposition only
[…]
```

2. Modify 20.20.6.2 [format.arg.store] as indicated:

```
namespace std {
  template<class Context, class... Args>
  structclass format-arg-store { // exposition only
    array<basic_format_arg<Context>, sizeof...(Args)> args; // exposition only
  };
}
```

[2021-05-18; Tim updates wording.]

[2021-05-20; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 20.20.1 [format.syn], header `<format>` synopsis, as indicated:

```
[…]
// 20.20.6.2 [format.arg.store], class template format-arg-store
template<class Context, class... Args> structclass format-arg-store; // exposition only

template<class Context = format_context, class... Args>
  format-arg-store<Context, Args...>
    make_format_args(const Args&... fmt_args);
[…]
```

2. Modify 20.20.6.2 [format.arg.store] as indicated:

```
namespace std {
  template<class Context, class... Args>
  structclass format-arg-store { // exposition only
    array<basic_format_arg<Context>, sizeof...(Args)> args; // exposition only
  };
}
```

-1- An instance of *`format-arg-store`* stores formatting arguments.

```
template<class Context = format_context, class... Args>
  format-arg-store<Context, Args...> make_format_args(const Args&... fmt_args);
```

-2- *Preconditions*: The type `typename Context::template formatter_type<T_i>` meets the *Formatter* requirements (20.20.5.1 [formatter.requirements]) for each T_i in `Args`.

-3- *Returns*: An object of type *`format-arg-store`*`<Context, Args...>` whose `args` data member is initialized with `{basic_format_arg<Context>(fmt_args)...}`.

3546. `common_iterator`'s *`postfix-proxy`* is not quite right

Section: 23.5.4.5 [common.iter.nav] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-04-23 **Last modified:** 2021-05-17

Priority: Not Prioritized

Discussion:

P2259R1 modeled `common_iterator::operator++(int)`'s *`postfix-proxy`* class on the existing proxy class used by `common_iterator::operator->`, but in doing so it overlooked two differences:

- `operator->`'s proxy is only used when `iter_reference_t<I>` is not a reference type; this is not the case for *`postfix-proxy`*;
- `operator->` returns a prvalue proxy, while `operator++`'s *`postfix-proxy`* is returned by (elidable) move, so the latter needs to require `iter_value_t<I>` to be `move_constructible`.

The proposed wording has been implemented and tested.

[2021-05-10; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

[2021-05-17; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 23.5.4.5 [common.iter.nav] as indicated:

```
decltype(auto) operator++(int);
```

-4- *Preconditions:* `holds_alternative<I>(v_)` is `true`.

-5- *Effects:* If `I` models `forward_iterator`, equivalent to:

```
common_iterator tmp = *this;
++*this;
return tmp;
```

Otherwise, if `requires (I& i) { { *i++ } -> can-reference; }` is `true` or `constructible_from<iter_value_t<I>, iter_reference_t<I>> && move constructible<iter value t<I>>` is `false`, equivalent to:

```
return get<I>(v_)++;
```

Otherwise, equivalent to:

```
postfix-proxy p(**this);
++*this;
return p;
```

where *`postfix-proxy`* is the exposition-only class:

```
class postfix-proxy {
  iter_value_t<I> keep_;
  postfix-proxy(iter_reference_t<I>&& x)
    : keep_(std::moveforward<iter reference t<I>>(x)) {}
public:
  const iter_value_t<I>& operator*() const {
    return keep_;
  }
};
```

3548. `shared_ptr` construction from `unique_ptr` should move (not copy) the deleter

Section: 20.11.3.2 [util.smartptr.shared.const] **Status:** Tentatively Ready **Submitter:** Thomas Köppe **Opened:** 2021-05-04 **Last modified:** 2021-05-17

Priority: Not Prioritized

View other active issues in [util.smartptr.shared.const].

View all other issues in [util.smartptr.shared.const].

Discussion:

The construction of a `shared_ptr` from a suitable `unique_ptr` rvalue `r` (which was last modified by LWG 2415, but not in ways relevant to this issue) calls for `shared_ptr(r.release(), r.get_deleter())` in the case where the deleter is not a reference.

This specification requires the deleter to be copyable, which seems like an unnecessary restriction. Note that the constructor `unique_ptr(unique_ptr&& u)` only requires `u`'s deleter to be *Cpp17MoveConstructible*. By analogy, the constructor `shared_ptr(unique_ptr<Y, D>&&)` should also require `D` to be only move-, not copy-constructible.

[2021-05-17; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 20.11.3.2 [util.smartptr.shared.const] as indicated:

```
template<class Y, class D> shared_ptr(unique_ptr<Y, D>&& r);
```

-28- *Constraints:* `Y*` is compatible with `T*` and `unique_ptr<Y, D>::pointer` is convertible to `element_type*`.

-29- *Effects:* If `r.get() == nullptr`, equivalent to `shared_ptr()`. Otherwise, if `D` is not a reference type, equivalent to `shared_ptr(r.release(), std::move(r.get_deleter()))`. Otherwise, equivalent to `shared_ptr(r.release(), ref(r.get_deleter()))`. If an exception is thrown, the constructor has no effect.

3549. `view_interface` is overspecified to derive from `view_base`

Section: 24.5.3 [view.interface], 24.4.4 [range.view] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-05-09 **Last modified:** 2021-05-26

Priority: 2

Discussion:

`view_interface<D>` is currently specified to publicly derive from `view_base`. This derivation requires unnecessary padding in range adaptors like `reverse_view<V>`: the `view_base` subobject of the `reverse_view` is required to reside at a different address than the `view_base` subobject of `V` (assuming that `V` similarly opted into being a view through derivation from `view_interface` or `view_base`).

This appears to be a case of overspecification: we want public and unambiguous derivation from `view_interface` to make the class a `view`; the exact mechanism of how that opt-in is accomplished should have been left as an implementation detail.

The proposed resolution below has been implemented on top of libstdc++ master and passes its ranges testsuite, with the exception of a test that checks for the size of various range adaptors (and therefore asserts the existence of `view_base`-induced padding).

[2021-05-17; Reflector poll]

Priority set to 2.

[2021-05-26; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 24.4.4 [range.view] as indicated:

```
template<class T>
  inline constexpr bool is-derived-from-view-interface = see below;   // exposition only
template<class T>
  inline constexpr bool enable_view = derived_from<T, view_base> || is-derived-from-view-interface<T>;
```

-?- For a type `T`, *`is-derived-from-view-interface`*`<T>` is `true` if and only if `T` has exactly one public base class `view_interface<U>` for some type `U` and `T` has no base classes of type `view_interface<V>` for any other type `V`.

-5- Remarks: Pursuant to 16.4.5.2.1 [namespace.std], users may specialize `enable_view` to `true` for cv-unqualified program-defined types which model `view`, and `false` for types which do not. Such specializations shall be usable in constant expressions (7.7 [expr.const]) and have type `const bool`.

2. Modify 24.5.3.1 [view.interface.general] as indicated:

```
namespace std::ranges {
  template<class D>
    requires is_class_v<D> && same_as<D, remove_cv_t<D>>
  class view_interface : public view_base {

  […]

  };
}
```

3551. `borrowed_{iterator,subrange}_t` are overspecified

Section: 24.5.5 [range.dangling] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-05-12 **Last modified:** 2021-05-20

Priority: Not Prioritized

Discussion:

As discussed in P1715R0, there are ways to implement something equivalent to `std::conditional_t` that are better for compile times. However, `borrowed_{iterator,subrange}_t` are currently specified to use `conditional_t`, and this appears to be user-observable due to the transparency of alias templates. We should simply specify the desired result and leave the actual definition to the implementation.

[2021-05-20; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 24.2 [ranges.syn], header `<ranges>` synopsis, as indicated:

```
  […]
namespace std::ranges {
  […]
  // 24.5.5 [range.dangling], dangling iterator handling
  struct dangling;

  template<range R>
    using borrowed_iterator_t = see below~~conditional_t<borrowed_range<R>, iterator_t<R>, dangling>~~;

  template<range R>
    using borrowed_subrange_t = see below
      ~~conditional_t<borrowed_range<R>, subrange<iterator_t<R>>, dangling>~~;

  […]
}
```

2. Modify 24.5.5 [range.dangling] as indicated:

> -1- The tag type `dangling` is used together with the template aliases `borrowed_iterator_t` and `borrowed_subrange_t`. When an algorithm that typically returns an iterator into, or a subrange of, a range argument is called with an rvalue range argument that does not model `borrowed_range` (24.4.2 [range.range]), the return value possibly refers to a range whose lifetime has ended. In such cases, the tag type `dangling` is returned instead of an iterator or subrange.
>
> ```
> namespace std::ranges {
>   struct dangling {
>     […]
>   };
> }
> ```
>
> -2- [*Example 1:* […] — *end example*]
>
> -?- For a type `R` that models `range`:
>
> (?.1) — if `R` models `borrowed_range`, then `borrowed_iterator_t<R>` denotes `iterator_t<R>`, and `borrowed_subrange_t<R>` denotes `subrange<iterator_t<R>>`;
>
> (?.2) — otherwise, both `borrowed_iterator_t<R>` and `borrowed_subrange_t<R>` denote `dangling`.

3552. Parallel specialized memory algorithms should require forward iterators

Section: 20.10.2 [memory.syn] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-05-16 **Last modified:** 2021-05-20

Priority: Not Prioritized

View all other issues in [memory.syn].

Discussion:

The parallel versions of `uninitialized_{copy,move}{,_n}` are currently depicted as accepting input iterators for their source range. Similar to the non-uninitialized versions, they should require the source range to be at least forward.

[2021-05-20; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 20.10.2 [memory.syn], header `<memory>` synopsis, as indicated:

```
  […]
namespace std {
  […]
```

```
  template<class InputIterator, class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_copy(InputIterator first, InputIterator last,
                                              NoThrowForwardIterator result);
  template<class ExecutionPolicy, class InputForwardIterator, class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_copy(ExecutionPolicy&& exec, // see 25.3.5 [algorithms.parallel.overloads]
                                              InputForwardIterator first, InputForwardIterator last,
                                              NoThrowForwardIterator result);
  template<class InputIterator, class Size, class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_copy_n(InputIterator first, Size n,
                                                NoThrowForwardIterator result);
  template<class ExecutionPolicy, class InputForwardIterator, class Size, class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_copy_n(ExecutionPolicy&& exec, // see 25.3.5 [algorithms.parallel.overloads]
                                                InputForwardIterator first, Size n,
                                                NoThrowForwardIterator result);
  […]

  template<class InputIterator, class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_move(InputIterator first, InputIterator last,
                                              NoThrowForwardIterator result);
  template<class ExecutionPolicy, class InputForwardIterator, class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_move(ExecutionPolicy&& exec, // see 25.3.5 [algorithms.parallel.overloads]
                                              InputForwardIterator first, InputForwardIterator last,
                                              NoThrowForwardIterator result);
  template<class InputIterator, class Size, class NoThrowForwardIterator>
    pair<InputIterator, NoThrowForwardIterator>
      uninitialized_move_n(InputIterator first, Size n, NoThrowForwardIterator result);
  template<class ExecutionPolicy, class InputForwardIterator, class Size, class NoThrowForwardIterator>
    pair<InputForwardIterator, NoThrowForwardIterator>
      uninitialized_move_n(ExecutionPolicy&& exec, // see 25.3.5 [algorithms.parallel.overloads]
                           InputForwardIterator first, Size n, NoThrowForwardIterator result);

  […]
}
```

3553. Useless constraint in `split_view::outer-iterator::value_type::begin()`

Section: 24.7.12.4 [range.split.outer.value] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2021-05-18 **Last modified:** 2021-05-26

Priority: Not Prioritized

View all other issues in [range.split.outer.value].

Discussion:

The `copyable` constraint in `split_view::outer-iterator::value_type::begin()` is useless because *`outer-iterator`* always satisfies `copyable`.

When `V` does not model `forward_range`, the *`outer-iterator`* only contains a pointer, so it is obviously copyable; When `V` is a `forward_range`, the `iterator_t<Base>` is a `forward_iterator`, so it is copyable, which makes *`outer-iterator`* also copyable.

Suggested resolution: Remove the no-`const begin()` and useless `copyable` constraint in 24.7.12.4 [range.split.outer.value].

[2021-05-26; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 24.7.12.4 [range.split.outer.value], class `split_view::outer-iterator::value_type` synopsis, as indicated:

```
    namespace std::ranges {
      template<input_range V, forward_range Pattern>
        requires view<V> && view<Pattern> &&
                 indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
                 (forward_range<V> || tiny-range<Pattern>)
      template<bool Const>
      struct split_view<V, Pattern>::outer-iterator<Const>::value_type
        : view_interface<value_type> {
      private:
        outer-iterator i_ = outer-iterator(); // exposition only
      public:
        value_type() = default;
        constexpr explicit value_type(outer-iterator i);
        constexpr inner-iterator<Const> begin() const requires copyable<outer-iterator>;
        constexpr inner-iterator<Const> begin() requires (!copyable<outer-iterator>);
        constexpr default_sentinel_t end() const;
      };
    }

[…]

constexpr inner-iterator<Const> begin() const requires copyable<outer-iterator>;

    -2- Effects: Equivalent to: return inner-iterator<Const>{i_};
```

~~constexpr *inner_iterator*<Const> begin() requires (!copyable<*outer_iterator*>);~~

~~*3- Effects:* Equivalent to: return *inner-iterator*<Const>{std::move(*i_*)};~~

[…]

3555. `{transform,elements}_view::iterator::iterator_concept` should consider const-qualification of the underlying range

Section: 24.7.6.3 [range.transform.iterator], 24.7.16.3 [range.elements.iterator] **Status:** Tentatively Ready **Submitter:** Tim Song **Opened:** 2021-05-23 **Last modified:** 2021-05-26

Priority: Not Prioritized

View other active issues in [range.transform.iterator].

View all other issues in [range.transform.iterator].

Discussion:

`transform_view::iterator<true>::iterator_concept` and `elements_view::iterator<true>::iterator_concept` (i.e., the const versions) are currently specified as looking at the properties of `V` (i.e., the underlying view without const), while the actual iterator operations are all correctly specified as using *`Base`* (which includes the const). `iterator_concept` should do so too.

The proposed resolution has been implemented and tested on top of libstdc++.

[2021-05-26; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4885.

1. Modify 24.7.6.3 [range.transform.iterator] as indicated:

-1- `iterator::iterator_concept` is defined as follows:

(1.1) — If ~~*V*~~*`Base`* models `random_access_range`, then `iterator_concept` denotes `random_access_iterator_tag`.

(1.2) — Otherwise, if ~~*V*~~*`Base`* models `bidirectional_range`, then `iterator_concept` denotes `bidirectional_iterator_tag`.

(1.3) — Otherwise, if ~~*V*~~*`Base`* models `forward_range`, then `iterator_concept` denotes `forward_iterator_tag`.

(1.4) — Otherwise, `iterator_concept` denotes `input_iterator_tag`.

2. Modify 24.7.16.3 [range.elements.iterator] as indicated:

-1- The member *typedef-name* `iterator_concept` is defined as follows:

(1.1) — If ~~*V*~~*`Base`* models `random_access_range`, then `iterator_concept` denotes `random_access_iterator_tag`.

(1.2) — Otherwise, if ~~*V*~~*`Base`* models `bidirectional_range`, then `iterator_concept` denotes `bidirectional_iterator_tag`.

(1.3) — Otherwise, if ~~*V*~~*`Base`* models `forward_range`, then `iterator_concept` denotes `forward_iterator_tag`.

(1.4) — Otherwise, `iterator_concept` denotes `input_iterator_tag`.